

Recursion Enabled: Improved Cryptanalysis of the Permuted Kernel Problem

Alessandro Budroni¹, Marco Defranceschi^{1,2*}, and Federico Pintore²

¹ Technology Innovation Institute, UAE
{alessandro.budroni,marco.defranceschi}@tii.ae

² University of Trento, Italy
federico.pintore@unitn.it

Abstract. The Permuted Kernel Problem (PKP) is a computational problem for linear codes over finite fields that has emerged as a promising hard problem for constructing post-quantum cryptographic schemes, with its main application found in the digital signature scheme PERK, submitted to the NIST standardization process for quantum-secure additional signatures. Upon reviewing the first version of PERK, NIST recommended further research on the concrete complexity of PKP. In this work, we follow this recommendation and investigate algorithmic improvements to the known methods for solving PKP. Specifically, we build upon the state-of-the-art work of Santini, Baldi, and Chiaraluce (IEEE Trans. Inf. Theory, 2024), and introduce a new algorithm that outperforms it over a wide range of parameters, yielding double-digit bit reductions in estimated complexity on representative instances. Nevertheless, our analysis shows that these improvements do not affect the parameter-set choices in PERK, thereby reinforcing confidence in its security.

Keywords: Permuted Kernel Problem · Code-based Cryptography · Post-quantum Cryptography · PERK

1 Introduction

Following Shor’s seminal work [27], which introduced a quantum algorithm capable of solving the hard mathematical problems underlying much of classical cryptography in polynomial time, the cryptographic community began to seek alternative hardness assumptions that would withstand quantum adversaries. This effort gave birth to the field of Post-quantum Cryptography.

In 2017, the National Institute of Standards and Technology (NIST) launched a call for post-quantum schemes [18]. Three out of the five schemes selected

*This work was conducted during Marco Defranceschi’s internship at the Technology Innovation Institute, Abu Dhabi.

for standardization are based on assumptions on algebraic lattices [25,17,21], reflecting the maturity reached by cryptosystems based on this family of problems. However, with the subsequent NIST “on-ramp” call for additional digital signature [19], NIST emphasized the importance of diversifying beyond lattices, encouraging the study of alternative quantum-resistant assumptions. In this context, the Permuted Kernel Problem (PKP), originally introduced by Shamir in 1989 [26] to construct an identification scheme, has resurfaced as a promising candidate for building digital signatures. Among the proposals submitted to the NIST “on-ramp” call for additional signatures, PERK [1] is a digital signature scheme built on the hardness of PKP. PERK successfully advanced through the first evaluation phase and is now in the second round of the NIST process for the selection of additional signatures to standardize.

Motivation and contribution. In its report on the first round of the additional signatures selection process, where candidates advancing to the second round are enumerated [3, Sec. 3.6], NIST explicitly highlighted the need for further research on the concrete complexity of PKP, underlining the importance of assessing the long-term security of the parameter choice in PERK. In this work, we take up this challenge and present new advances in the cryptanalysis of PKP, focusing on its concrete complexity. Specifically, we introduce a new algorithm that improves upon the state of the art across several parameter sets.

Related work. Shamir was the first one to construct a cryptographic scheme based on PKP [26]. More recently, a series of digital signature schemes relying on the hardness of PKP have been proposed [10,8,7,11,4]. However, the most prominent instantiation is represented by PERK [1]. Specifically, PERK version 1.0, submitted to the first round of the NIST process for additional signatures, relies on a variant of PKP that can nonetheless be solved by any algorithm solving the standard PKP. More recently, version 2.1 of PERK has been made public [2], this time relying on the pure PKP over a finite field of characteristic two.

Shamir was also the first one to provide some considerations on the security of PKP [26]. In particular, he observed that a reduction from the 3-Partition problem implies that the mono-dimensional version of PKP is NP-complete [12]. The NP-completeness of the multi-dimensional case was later proved in [6]. Since Shamir’s original work, several works have examined the hardness of PKP [13,5,20,14,16], progressively refining the understanding of the computational complexity of the problem. More recently, Koussa, Macario-Rat and Patarin (KMP) [15] proposed a *meet-in-the-middle* algorithm, which was subsequently improved by Santini, Baldi, and Chiaraluce (SBC) [23,24] through the introduction of a preprocessing (*filtering*) step. The latter currently represents the state of the art in solving PKP.

1.1 Overview of the Contribution

The Permuted Kernel Problem. Informally, the Permuted Kernel Problem (PKP) consists in finding a permutation π such that

$$\pi(\mathbf{V})\mathbf{H}^\top = \mathbf{0},$$

where $\mathbf{H} \in \mathbb{F}_q^{r \times n}$ and $\mathbf{V} \in \mathbb{F}_q^{\ell \times n}$ are public matrices. An equivalent inhomogeneous version of the problem, denoted by IPKP, asks for a permutation π satisfying

$$\pi(\mathbf{V})\mathbf{H}^\top = \mathbf{E}, \tag{1}$$

where $\mathbf{E} \in \mathbb{F}_q^{\ell \times r}$ is also public. Typically, the parameters n, r, ℓ , and q are chosen so that the permutation π is unique with high probability.

Key contributions. In one of its subroutines, the SBC algorithm employs an Information Set Decoding (ISD) procedure to find a subcode of the code generated by $\mathbf{H}$ having dimension $d \leq r$ and small support $w \leq n$, that is, a subcode with a generator matrix with $n - w$ zero columns. The algorithm then performs a *meet-in-the-middle* style routine over this subcode to filter potential candidates for the permutation π .

Our first algorithmic improvement stems from the observation that, by disregarding the zero columns in the generator matrix of the subcode with support size w , one obtains a subinstance of IPKP where the corresponding public matrix $\tilde{\mathbf{H}}$ has size $d \times w$. In other words, the *filtering* subroutine of SBC can be interpreted as solving smaller IPKP instances. This suggests that SBC could be applied recursively, since SBC solves IPKP (and not only PKP). However, this recursive approach is generally inefficient or inapplicable unless one considers another key observation of this work, that is, the dual form of Equation (1). Indeed, we notice that

$$\pi^{-1}(\mathbf{H})\mathbf{V}^\top = \mathbf{E}^\top,$$

can be seen as an instance of IPKP with secret permutation π^{-1} (the inverse of π), and we find that applying SBC recursively leads to improvements only when the subinstance is considered in its dual version. Moreover, enabling such recursion is non-trivial as the subinstance may admit no solution or multiple solutions. To address this, we revisited and extended SBC, introducing several subtle modifications to make recursive application feasible.

Building on the observations above, we introduce a new algorithm that either outperforms or matches SBC. Specifically, we find improvements for parameter sets where q is relatively small. Most importantly, depending on the code rate $R := r/n$, our algorithm is faster also for very small values of ℓ , that is the typical setting in cryptographic applications. As an example, for $(n, r, \ell, q) = (112, 54, 3, 16)$, our algorithm reduces the estimated complexity from 255 to 243 bits, corresponding to an improvement of 12 bits. Similarly, for $(n, r, \ell, q) = (118, 83, 1, 223)$, we observe a reduction from 199 to 189 bits, i.e., an improvement of 10 bits.

Implications for PERK. Despite significantly improving upon SBC for a wide range of parameters, our newly proposed algorithm does not affect the security estimates of either PERK 1.0 or PERK 2.1. In the former case, the parameter sets employ a very large field size q , which prevents our algorithmic improvements from being effective. In the latter, the combination of the code rate R and the choice $\ell = 1$ places the parameters in a regime where SBC remains the optimal solving algorithm.

Artifacts. We make our scripts used to compute the complexities publicly available [9].

Organization. In Section 2, we provide the necessary preliminaries and background. Section 3 reviews the KMP and SBC algorithms, serving as essential background for introducing our contribution in Section 4. Finally, we analyze our results and evaluate their impact on PERK in Section 5.

2 Preliminaries

Notation. Let $n \in \mathbb{N}^*$. We define $[n] := \{1, \dots, n\}$. If $I \subseteq [n]$, we denote by $\neg_n I$ its complementary $[n] \setminus I$. We use capital bold letters for matrices and lower-case bold letters for vectors. When the base field is clear from the context, the $n \times n$ identity matrix is denoted by $\mathbf{I}_n$. All vectors are row-vectors if not stated otherwise. Given a vector $\mathbf{v}$ we denote by v_i its i -th entry. Similarly, for an $m \times n$ matrix $\mathbf{A}$, with $m \in \mathbb{N}^*$, we denote its j -th column by $\mathbf{a}_j$ and its entry in the i -th row and j -th column by $a_{i,j}$. Given a set of indices $I \subseteq [n]$, $\mathbf{v}_I$ is the projection of the length- n vector $\mathbf{v}$ onto the indices in I , i.e. $\mathbf{v}_I = (v_i)_{i \in I}$. We extend this to matrices by letting $\mathbf{A}_I$ be the submatrix formed by the columns of $\mathbf{A}$ indexed by I . Similarly, we denote by $\mathbf{A}^J$ the submatrix obtained by taking the rows of $\mathbf{A}$ indexed by J , if $J \subseteq [m]$. The transpose of a matrix $\mathbf{A}$ is denoted by $\mathbf{A}^\top$.

Let $\mathbb{F}_q$ be a finite field of size q , with q a prime power. We denote by $\text{GL}_n(\mathbb{F}_q)$ the linear group of invertible $n \times n$ matrices over $\mathbb{F}_q$ and by $\text{GL}_{k,n}(\mathbb{F}_q)$ the set of $k \times n$ (with $k \leq n$) matrices over $\mathbb{F}_q$ with full rank k .

We denote by $\text{Sym}(n)$ the set of all permutations of $[n]$. For a permutation $\pi \in \text{Sym}(n)$ and $\mathbf{A} \in \mathbb{F}_q^{m \times n}$, we write $\pi(\mathbf{A})$ to denote the matrix obtained by permuting the columns of $\mathbf{A}$ according to π , that is the matrix $\mathbf{A}\mathbf{P}$ where $\mathbf{P} \in \mathbb{F}_q^{n \times n}$ is the permutation matrix associated to π .

Given a matrix $\mathbf{A} \in \text{GL}_{k,n}(\mathbb{F}_q)$ we say that a set of indices $I \subseteq [n]$ with cardinality k is an information set for $\mathbf{A}$ if $\mathbf{A}_I$ is invertible. When this is the case, we denote by $\text{RREF}_I(\mathbf{A})$ the *Row Reduced Echelon Form* of $\mathbf{A}$ w.r.t. to the set I , that is $\mathbf{A}_I^{-1}\mathbf{A}$. In addition, we denote by $\text{RREF}^*_I(\mathbf{A})$ the pair composed of $\text{RREF}_I(\mathbf{A})$ and the matrix $\mathbf{A}_I^{-1}$.

For a matrix $\mathbf{A}$ with n columns and a non-zero natural number $x \leq n$, we define $\mathcal{S}_x(\mathbf{A})$ as the set of all matrices with x columns, each of them picked from those of $\mathbf{A}$. It is easy to see that, if $\mathbf{A}$ has no repeated columns (we show in Section 4.2 that this might not be always the case), $|\mathcal{S}_x(\mathbf{A})| = \frac{n!}{(n-x)!}$. We call $\bar{\mathcal{S}}_x(\mathbf{A})$ the set obtained by identifying elements in $\mathcal{S}_x(\mathbf{A})$ up to column permutation, that is, the set of all the subsets of x columns of $\mathbf{A}$. Therefore, $|\bar{\mathcal{S}}_x(\mathbf{A})| = \frac{|\mathcal{S}_x(\mathbf{A})|}{x!} = \binom{n}{x}$.

With an abuse of notation, for two matrices $\mathbf{A} \in \mathbb{F}_q^{m \times n}$ and $\mathbf{B} \in \mathbb{F}_q^{n \times \ell}$ we say that $\mathbf{B} \in \ker(\mathbf{A})$ if $\mathbf{AB} = \mathbf{0}$.

Finally, we say that an event occurs with "high probability" if it happens with probability $1 - o(1)$.

Linear Codes. An $[n, k]$ -linear code $\mathcal{C}$ over a finite field $\mathbb{F}_q$ is a subspace, of dimension k , of $\mathbb{F}_q^n$. $\mathcal{C}$ can be represented via a basis $\mathbf{G} \in \mathbb{F}_q^{k \times n}$, called *generator matrix*, or by a *parity-check matrix* $\mathbf{H} \in \mathbb{F}_q^{(n-k) \times n}$ satisfying $\mathbf{H}\mathbf{c}^\top = \mathbf{0} \Leftrightarrow \mathbf{c} \in \mathcal{C}$. The linear code generated by (the rows of) $\mathbf{H}$ is called the dual code of $\mathcal{C}$ and is denoted by $\mathcal{C}^\perp$. We refer to n as the *length* of $\mathcal{C}$ and to the quantity $R := k/n$ as the *code rate*. Given a codeword $\mathbf{c} \in \mathcal{C}$, we define its support as the set of indices of its nonzero entries, i.e. $\text{Supp}(\mathbf{c}) := \{i \in [n] : \mathbf{c}_i \neq 0\}$. We also define the support of a linear code $\mathcal{C}$ as the union of the supports of its codewords, that is $\text{Supp}(\mathcal{C}) := \{i \in [n] : \exists \mathbf{c} \in \mathcal{C} \text{ s.t. } \mathbf{c}_i \neq 0\}$. The Hamming weight $\text{wt}(\mathbf{c}) := |\text{Supp}(\mathbf{c})|$ of $\mathbf{c}$ is the size of its support.

A subspace $\mathcal{C}'$ of a linear code $\mathcal{C}$ over $\mathbb{F}_q$ is said to be a subcode of $\mathcal{C}$. If $\mathcal{C}$ has dimension k , the number of its subcodes having dimension $k' \leq k$ is given by $\begin{bmatrix} k \\ k' \end{bmatrix}_q = \prod_{i=0}^{k'-1} \frac{1-q^{k-i}}{1-q^{i+1}}$. In our complexity estimates, we make use of the approximation used in [24], i.e.

$$\frac{\begin{bmatrix} \omega n \\ d \end{bmatrix}_q}{\begin{bmatrix} n \\ d \end{bmatrix}_q} \approx q^{-d(1-\omega)n} \quad (2)$$

for $d \in \mathbb{N}$ and $\omega \in [0, 1]$.

Counting the number of operations. Coherently with the PKP literature, in our complexity analysis we consider as elementary operations both matrix multiplications and list operations. Therefore, when we say "operations", we refer to such operations and not to finite-field additions/multiplications.

Subcodes with small support. Given a $[n, k]$ -linear code over $\mathbb{F}_q$, we follow [23, Thm. IV.1] and estimate the number of subcodes with support size $w \leq n$ and dimension $d \leq k$ with

$$N_{n,k}(w, d) = \binom{n}{w} (q^d - 1)^{w-d} \frac{\begin{bmatrix} k \\ d \end{bmatrix}_q}{\begin{bmatrix} n \\ d \end{bmatrix}_q}.$$

In [24, Prop. 2], it is shown the existence of an adaptation of Prange's ISD algorithm ([24, Alg. 3]) that takes in input a linear code $\mathcal{C} \subseteq \mathbb{F}_q^n$ of dimension k

and two other integers $w, d \in \mathbb{N}$ such that $w \leq n + d - k$ and returns a subcode of $\mathcal{C}$ with dimension d and support size w . The average running time of the algorithm is

$$T_{\text{ISD}}^{(d)}(n, k, w) = \mathcal{O} \left(\frac{k^3 + \binom{k}{d}}{p^{(d)}(n, k, w)} \right) \quad (3)$$

$$\text{with } p^{(d)}(n, k, w) = \min \left\{ \frac{\binom{w}{d} \binom{n-w}{k-d}}{\binom{n}{k}} N_{n,k}(w, d), 1 \right\}.$$

2.1 The Permuted Kernel Problem

We now introduce the Permuted Kernel Problem (PKP), the central object of our study. We consider it directly in its search version, that is, under the assumption that a solution exists, as is the case in the setting of PERK ([1]).

Definition 1 (Permuted Kernel Problem). *Let $\mathbf{H} \in \mathbb{F}_q^{m \times n}$ be a random matrix and $\mathbf{V} \in \mathbb{F}_q^{\ell \times n}$, with $\ell \leq n - m$, be a random matrix for which there exists a permutation $\pi \in \text{Sym}(n)$ such that $\pi(\mathbf{V})\mathbf{H}^\top = \mathbf{0}$. The Permuted Kernel Problem (PKP) is to recover π from $\mathbf{H}$ and $\mathbf{V}$.*

With this notation, we say that a PKP instance has parameters (n, m, q, ℓ) . Note that in Definition 1 a solution exists by construction. However, in the following sections, we also consider instances that may not admit any solution. In such cases, we rely on an exhaustive search approach for solving them, which allows to detect whenever no solution exists. We nevertheless continue to refer to these as “PKP instances,” even if they admit no solutions.

Remark 1. In contrast to [24], we restrict from the beginning to the case $\ell \leq n - m$. We show that this is actually not a restriction. First, notice that we can consider the matrices $\mathbf{V}$ and $\mathbf{H}$ to be full rank, since if that is not the case we can obtain an equivalent instance (i.e. they admit the same solutions) by removing the redundant rows. Such instance is equivalent to the starting one because the redundant rows introduce equations that are always verified (the other direction is easy). Under the assumption of full-rank matrices, the case $\ell > n - m$ is not possible. This can be easily seen considering the kernel dimension: consider $\mathbf{H} \in \text{GL}_{m,n}(\mathbb{F}_q)$, $\mathbf{V} \in \text{GL}_{\ell,n}(\mathbb{F}_q)$ and $\pi \in \text{Sym}(n)$ such that

$$\pi(\mathbf{V})\mathbf{H}^\top = \mathbf{0},$$

which is equivalent to requiring that $\mathbf{H}^\top \in \ker(\pi(\mathbf{V}))$. If the matrices are full-rank, we have that $\dim(\ker(\pi(\mathbf{V}))) = n - \text{rank}(\pi(\mathbf{V})) = n - \ell$, which implies that $m \leq n - \ell$.

On the expected number of solutions. If π is a solution to the instance, its corresponding subspace must be orthogonal to the one generated by the rows of

$\mathbf{H}$. Thus, the space generated by the rows of $\pi(\mathbf{V})$ must lie in the orthogonal complement of the subspace generated by the rows of $\mathbf{H}$, which has dimension $n - m$. Therefore, the probability that a random ℓ -dimensional subspace of $\mathbb{F}_q^n$ is orthogonal to the space generated by $\mathbf{H}$ is $\frac{\begin{bmatrix} n-m \\ \ell \end{bmatrix}_q}{\begin{bmatrix} n \\ \ell \end{bmatrix}_q}$. The expected number of solutions is $n! \frac{\begin{bmatrix} n-m \\ \ell \end{bmatrix}_q}{\begin{bmatrix} n \\ \ell \end{bmatrix}_q}$. By putting $\omega = (1 - \frac{m}{n})$ and $d = \ell$ in Equation (2), we can approximate this quantity with $n!q^{-\ell m}$. Coherently with the literature on PKP [15,24], we consider instances where only one solution is expected, which is guaranteed by the following conditions (see [24]):

- i) $\text{rank}(\mathbf{V}) = \ell$ and $\text{rank}(\mathbf{H}) = m$;
- ii) $\mathbf{V}$ and $\mathbf{H}$ have no repeated columns;
- iii) $q^\ell \geq n$ and $q^m \geq n$;
- iv) $n! \frac{\begin{bmatrix} n-m \\ \ell \end{bmatrix}_q}{\begin{bmatrix} n \\ \ell \end{bmatrix}_q} < 1$.

Condition iii) is necessary because, for every column of $\mathbf{V}$ and $\mathbf{H}$, there are q^ℓ and q^m possible choices, respectively. If this condition does not hold, then either $\mathbf{V}$ or $\mathbf{H}$ must contain repeated columns. Condition iv) ensures that, in expectation, there exists only one solution—namely, the one introduced by construction. We remark that, from a purely brute-force approach point of view, instances admitting a unique solution are the hardest to solve. However, this is not necessarily the case for more advanced algorithms [15,24], whose complexity might grow with the expected number of solutions. Nevertheless, this choice is conservative in our case, since our algorithm introduced in Section 4 significantly benefits from having smaller field sizes q , which typically occurs when multiple solutions are expected. An in-depth analysis of the number of solutions to a random PKP instance was recently given in [22].

2.2 Basic approach to solve PKP

We show now how to exploit the kernel dimension to reduce the computational complexity of a brute force attack on PKP, which in principle is $n!$. Before doing that, we introduce the following variant of the problem.

Definition 2 (Inhomogeneous PKP). *Let $\mathbf{H} \in \mathbb{F}_q^{r \times n}$ be a random matrix. Let $\mathbf{V} \in \mathbb{F}_q^{\ell \times n}$ and $\mathbf{E} \in \mathbb{F}_q^{\ell \times r}$, with $\ell \leq n - r$, be random matrices for which there exists a permutation $\pi \in \text{Sym}(n)$ such that $\pi(\mathbf{V})\mathbf{H}^\top = \mathbf{E}$. The Inhomogeneous Permuted Kernel Problem (IPKP) is to recover π from $\mathbf{H}$, $\mathbf{V}$ and $\mathbf{E}$.*

Analogously as for PKP, we say that an IPKP instance has parameters (n, r, q, ℓ) and we refer to IPKP instances even in cases that may not admit any solution. For

the sake of easy presentation, we also denote by $\pi(\mathbf{V})\mathbf{H}^\top = \mathbf{E}$ an IPKP instance defined by the matrices $\mathbf{H}$, $\mathbf{V}$ and $\mathbf{E}$. Moreover, we refer to IPKP instances even in cases in which the matrices are not completely random, as we will deal with instances in which the matrix $\mathbf{E}$ and the last row of $\mathbf{H}$ are fixed. We do this because our original goal is to solve a (homogeneous) PKP instance, which we show now to be equivalent to an IPKP instance with such properties.

If $\mathbf{H} \in \mathbb{F}_q^{m \times n}$, $\mathbf{V} \in \mathbb{F}_q^{\ell \times n}$ and $\pi(\mathbf{V})\mathbf{H}^\top = \mathbf{0}$ for some $\pi \in \text{Sym}(n)$, then if we define $\mathbf{H}^{\text{ext}} = \begin{bmatrix} \mathbf{H} \\ (1, \dots, 1) \end{bmatrix}$, $\mathbf{E} = [\mathbf{0}_{\ell \times m} \sum_{i=1}^n \mathbf{v}_i]$ and $\tilde{\mathbf{V}} = \pi(\mathbf{V})$, we have that

$$\tilde{\mathbf{V}}\mathbf{H}^{\text{ext}\top} = [\tilde{\mathbf{V}}\mathbf{H}^\top \tilde{\mathbf{V}} \cdot (1, \dots, 1)^\top] = [\mathbf{0}_{\ell \times m} \sum_{i=1}^n \tilde{\mathbf{v}}_i] = \mathbf{E}. \quad (4)$$

This means that $\pi \in \text{Sym}(n)$ also solves the IPKP instance defined by the matrices $\mathbf{H}^{\text{ext}}$, $\mathbf{V}$ and $\mathbf{E}$. The other direction follows because, if $\pi \in \text{Sym}(n)$ satisfies Equation (4), then it holds that $\tilde{\mathbf{V}}(\mathbf{H}^{\text{ext}\{1, \dots, m\}})^\top = \mathbf{E}_{\{1, \dots, m\}} = \mathbf{0}_{\ell \times m}$, which is the equation defining the starting PKP instance. This is useful because we now show that to solve an (n, r, q, ℓ) -IPKP instance it suffices to guess the action of π on $n - r$ coordinates. The same applies to PKP. However, if we want to solve an (n, m, q, ℓ) -PKP instance, it is more convenient to solve the equivalent $(n, m + 1, q, \ell)$ -IPKP instance, since we need to guess the action of π on one coordinate less.

From now on, we refer to $\mathbf{H}$ as the extended parity check matrix.

Remark 2. In the same way as above, it is also possible to extend the parity check matrix in IPKP instances. In our newly-proposed algorithm we make use of this, see, for example, Line 7 and Line 15 in Algorithm 7. Notice, however, that we can not extend the same parity check matrix more than once, since doing this would not increase the rank.

Basic solver for IPKP. Let us consider now an (n, r, q, ℓ) -IPKP instance $(\mathbf{H}, \mathbf{V}, \mathbf{E})$ for which there exists $\pi \in \text{Sym}(n)$ such that $\pi(\mathbf{V})\mathbf{H}^\top = \mathbf{E}$. If we suppose that $J = \{n - r + 1, \dots, n\}$ is an information set for $\mathbf{H}$ and let $[\mathbf{U} \ \mathbf{I}_r]$, $\mathbf{S} = \text{RREF}^*_J(\mathbf{H})$, then $[\tilde{\mathbf{V}}_{\neg_n J} \ \tilde{\mathbf{V}}_J] (\mathbf{S}\mathbf{H})^\top = \mathbf{E}\mathbf{S}^\top$. This is equivalent to

$$\tilde{\mathbf{V}}_{\neg_n J} \mathbf{U}^\top + \tilde{\mathbf{V}}_J = \mathbf{E}\mathbf{S}^\top \Leftrightarrow \tilde{\mathbf{V}}_J = \mathbf{E}\mathbf{S}^\top - \tilde{\mathbf{V}}_{\neg_n J} \mathbf{U}^\top.$$

Finally, note that if J' is another information set for $\mathbf{H}$, it holds that

$$\tilde{\mathbf{V}}_{J'} = (\mathbf{E} - \tilde{\mathbf{V}}_{\neg_n J'} (\mathbf{H}_{\neg_n J'})^\top) (\mathbf{H}_{J'})^{-\top}, \quad (5)$$

where $^{-\top}$ denotes the inverse transposal operator. The relation above provides an improved brute-force solver: the columns with indices in $\neg_n J'$ are guessed, while the others are deduced from Equation (5). This method to solve IPKP is formalized in Algorithm 1 and has a time complexity of $\mathcal{O}(\frac{n!}{r!})$. This is obtained because the *Row-Reduced-Echelon-Form* can be computed with one elementary operation, since computing it has the same cost as a matrix multiplication (that

is $\mathcal{O}(n^3)$ field operations). Therefore the cost of computing the RREF is absorbed by the $\mathcal{O}$ -notation. In Algorithm 1, for the sake of easy presentation, we assume that $\{n - r + 1, \dots, n\}$ is an information set for $\mathbf{H}$, but the reasoning works for every information set J by using Equation (5).

Algorithm 1 SolvelPKP($\mathbf{H}, \mathbf{V}, \mathbf{E}$)

Input: $\mathbf{H} \in \mathbb{F}_q^{r \times n}$, $\mathbf{V} \in \mathbb{F}_q^{\ell \times n}$, $\mathbf{E} \in \mathbb{F}_q^{\ell \times r}$
Output: $\pi \in \text{Sym}(n)$ s.t. $\pi(\mathbf{V})\mathbf{H}^\top = \mathbf{E}$
1: $[\mathbf{U} \ \mathbf{I}_r], \mathbf{S} \leftarrow \text{RREF}_{\{n-r+1, \dots, n\}}^*(\mathbf{H})$
2: Initialize an empty list $\mathcal{L}$
3: **for** $\tilde{\mathbf{V}}_1 \in \mathcal{S}_{n-r}(\mathbf{V})$ **do**
4: $\tilde{\mathbf{V}}_2 \leftarrow \mathbf{E}\mathbf{S}^\top - \tilde{\mathbf{V}}_1\mathbf{U}^\top$
5: **if** $[\tilde{\mathbf{V}}_1 \ \tilde{\mathbf{V}}_2] \in \mathcal{S}_n(\mathbf{V})$ **then**
6: Compute $\pi \in \text{Sym}(n)$ s.t. $[\tilde{\mathbf{V}}_1 \ \tilde{\mathbf{V}}_2] = \pi(\mathbf{V})$
7: **Return:** π

2.3 Working with the dual

Consider now the following (n, r, q, ℓ) -IPKP instance.

$$\pi(\mathbf{V})\mathbf{H}^\top = \mathbf{E}.$$

If we take the transpose we obtain the *dual* version of the problem (notice that, if $\mathbf{E} = \mathbf{0}$, the *dual* denomination becomes proper)

$$\pi^{-1}(\mathbf{H})\mathbf{V}^\top = \mathbf{E}^\top. \quad (6)$$

This can be seen as an IPKP instance with parameters (n, ℓ, q, r) . By means of Algorithm 1, we can solve this problem instance by guessing the action of π^{-1} on $n - \ell$ coordinates, that is with $\mathcal{O}(\frac{n!}{\ell!})$ operations. Finally note that we can extend $\mathbf{V}$ by adding a row of ones in the same way as for $\mathbf{H}$ and so the number of operations needed is $\mathcal{O}(\frac{n!}{(\ell+1)!})$.

Remark 3. As explained above, Algorithm 1 terminates in either $\mathcal{O}(\frac{n!}{r!})$ (here we consider $\mathbf{H}$ to have already been extended) operations or in $\mathcal{O}(\frac{n!}{(\ell+1)!})$ operations when considering the dual problem. Considering the dual problem shows that IPKP (the same applies to PKP) becomes easier as ℓ grows, since Algorithm 1 becomes faster.

The existing algorithms, that are presented in the next section, also become faster as ℓ grows, but for different reasons (their time complexity depends on ℓ in a different way). In this paper we show how increasing ℓ can affect the hardness of the problem in a more impactful way than in the existing algorithms.

3 Review of the Known Attacks to PKP

In this section, we review the main known algorithms to solve PKP. We focus in particular on the algorithms by Koussa, Macario-Rat and Patarin (KMP) [15], and Santini, Baldi and Chiaraluce (SBC) [24]. More precisely, for the KMP algorithm, we review its generalization to the multi-dimensional (i.e. $\ell > 1$) setting given in [24].

3.1 KMP algorithm

We focus on solving an IPKP instance defined by the matrices $\mathbf{H} \in \mathbb{F}_q^{r \times n}$, $\mathbf{V} \in \mathbb{F}_q^{\ell \times n}$, $\mathbf{E} \in \mathbb{F}_q^{\ell \times r}$, that is, we want to find $\pi \in \text{Sym}(n)$ such that $\pi(\mathbf{V})\mathbf{H}^\top = \mathbf{E}$. As in Section 2, let $\tilde{\mathbf{V}} = \pi(\mathbf{V})$. In a nutshell, the KMP algorithm performs a brute-force attack to guess the action of π on $n - r$ columns, but it does it by adopting a *meet-in-the-middle* approach, which leads to significant time-complexity improvements. More specifically, the algorithm aims at guessing the action of π on v columns, with $v > n - r$, by guessing the action on u_1 and u_2 (with $v = u_1 + u_2$) columns separately and then matching the guessed columns. Notice that this matching can be seen as a sieving phase. The optimal values for u_1, u_2 depend on the instance and so they will be two parameters of the algorithm.

Outline of the algorithm. As in Algorithm 1, $\mathbf{H}$ is put in systematic form with the identity matrix on the right, that is, we are assuming that $\{n-r+1, \dots, n\}$ is an information set for $\mathbf{H}$, but the reasoning works for every information set. Let $\mathbf{H}', \mathbf{M} = \text{RREF}^*_{\{n-r+1, \dots, n\}}(\mathbf{H})$. Then $\tilde{\mathbf{V}}\mathbf{H}^\top = \mathbf{E}$ if and only if $\tilde{\mathbf{V}}\mathbf{H}'^\top = \mathbf{E}\mathbf{M}^\top$. Let $\bar{\mathbf{H}} = \mathbf{H}'^{\{1, \dots, \tilde{r}\}}_{\{1, \dots, u_1+u_2\}}$ be the matrix obtained by taking the first $\tilde{r}$ rows and the first u_1+u_2 columns of $\mathbf{H}'$, with $\tilde{r} = u_1+u_2 - (n-r) \geq 1$. Let $\mathbf{M}' = \mathbf{M}^{\{1, \dots, \tilde{r}\}}$ (notice that $\mathbf{M}' \in \text{GL}_{\tilde{r}, r}(\mathbb{F}_q)$), so that $\bar{\mathbf{H}} = \mathbf{M}'\mathbf{H}_{\{1, \dots, u_1+u_2\}}$. Then, it holds that $\tilde{\mathbf{V}}_{\{1, \dots, u_1+u_2\}}\bar{\mathbf{H}}^\top = \mathbf{E}\mathbf{M}'^\top$. Indeed, $\tilde{\mathbf{V}}\mathbf{H}'^\top = \mathbf{E}\mathbf{M}^\top$ and $\mathbf{H}'$ can be written as $\mathbf{H}' = \begin{bmatrix} \bar{\mathbf{H}} & \mathbf{0}_{\tilde{r} \times n - (u_1+u_2)} \\ \mathbf{A} & \mathbf{I}_{n - (u_1+u_2)} \end{bmatrix}$ with $\mathbf{A} \in \mathbb{F}_q^{(r-\tilde{r}) \times (u_1+u_2)}$, so that

$$\begin{aligned} \tilde{\mathbf{V}}\mathbf{H}'^\top &= [\tilde{\mathbf{V}}_{\{1, \dots, u_1+u_2\}} \quad \tilde{\mathbf{V}}_{\{u_1+u_2+1, \dots, n\}}] \begin{bmatrix} \bar{\mathbf{H}} & \mathbf{0}_{\tilde{r} \times n - (u_1+u_2)} \\ \mathbf{A} & \mathbf{I}_{n - (u_1+u_2)} \end{bmatrix}^\top = \\ &= [\tilde{\mathbf{V}}_{\{1, \dots, u_1+u_2\}}\bar{\mathbf{H}}^\top \quad \tilde{\mathbf{V}}_{\{1, \dots, u_1+u_2\}}\mathbf{A}^\top + \tilde{\mathbf{V}}_{\{u_1+u_2+1, \dots, n\}}] = \mathbf{E}\mathbf{M}^\top. \end{aligned}$$

By definition of $\mathbf{M}'$, $(\mathbf{E}\mathbf{M}^\top)_{\{1, \dots, \tilde{r}\}} = \mathbf{E}\mathbf{M}'^\top$. We define $\bar{\mathbf{E}} = \mathbf{E}\mathbf{M}'^\top$ and so we have

$$\tilde{\mathbf{V}}_{\{1, \dots, u_1+u_2\}}\bar{\mathbf{H}}^\top = \bar{\mathbf{E}}. \quad (7)$$

Then, we call $\bar{\mathbf{H}}_1 = \bar{\mathbf{H}}_{\{1, \dots, u_1\}}$ and $\bar{\mathbf{H}}_2 = \bar{\mathbf{H}}_{\{u_1+1, \dots, u_1+u_2\}}$ and search for candidates for $\tilde{\mathbf{V}}_{\{1, \dots, u_1+u_2\}}$. For a visualization of these matrices see Fig. 1.

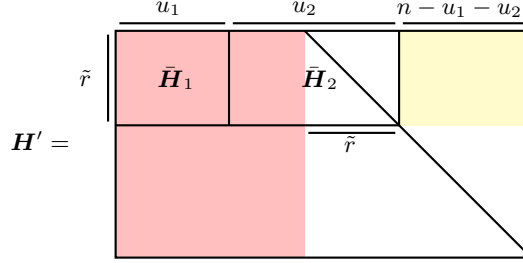


Fig. 1: Representation of the considered matrices, the yellow area is the zero matrix

Such candidates are found by building the following lists:

$$\begin{aligned}\mathcal{L}_1 &= \left\{ (\mathbf{X}, \mathbf{X} \bar{\mathbf{H}}_1^\top) \mid \mathbf{X} \in \mathcal{S}_{u_1}(\mathbf{V}) \right\}, \\ \mathcal{L}_2 &= \left\{ (\mathbf{Y}, \bar{\mathbf{E}} - \mathbf{Y} \bar{\mathbf{H}}_2^\top) \mid \mathbf{Y} \in \mathcal{S}_{u_2}(\mathbf{V}) \right\}.\end{aligned}\tag{8}$$

The algorithm proceeds by finding all *valid* collisions between $\mathcal{L}_1$ and $\mathcal{L}_2$, that is, finding pairs $(\mathbf{X}, \bar{\mathbf{X}}) \in \mathcal{L}_1$ and $(\mathbf{Y}, \bar{\mathbf{Y}}) \in \mathcal{L}_2$ s.t. $\bar{\mathbf{X}} = \bar{\mathbf{Y}}$ and $[\mathbf{X} \ \mathbf{Y}] \in \mathcal{S}_{u_1+u_2}(\mathbf{V})$ (therefore $[\mathbf{X} \ \mathbf{Y}]$ is a *valid* candidate for $\tilde{\mathbf{V}}_{\{1, \dots, u_1+u_2\}}$). We denote this operation of selecting all *valid* candidates by $\bowtie$, and call $\mathcal{L}$ the resulting list for $\tilde{\mathbf{V}}_{\{1, \dots, u_1+u_2\}}$, that is,

$$\mathcal{L} = \mathcal{L}_1 \bowtie \mathcal{L}_2 = \left\{ \mathbf{Z} \in \mathcal{S}_{u_1+u_2}(\mathbf{V}) \mid \mathbf{Z} \bar{\mathbf{H}}^\top = \bar{\mathbf{E}} \right\}.$$

From Equation (7) we have that $\tilde{\mathbf{V}}_{\{1, \dots, u_1+u_2\}} \in \mathcal{L}$ so, for every element in $\mathcal{L}$, we truncate it of $\tilde{r}$ columns and proceed as in Algorithm 1, and eventually find $\tilde{\mathbf{V}}$. Finally note that the requirement $u_1 + u_2 \geq n - r + 1$ is needed to have $\tilde{r} \geq 1$, so that it makes sense to look for collisions among matrices that have $\tilde{r}$ columns. The full algorithm is formalized in Algorithm 2.

Complexity of KMP. For the asymptotic analysis of the complexity of KMP we refer to [24]; here we are only interested in the finite regime, where its running time, measured in the number of matrix multiplications and list operations, is given by the following proposition.

Proposition 1 (Complexity of KMP [24, Prop. 5]). *Let $u_1, u_2 \in \mathbb{N}^*$ such that $n - r + 1 \leq u_1 + u_2 \leq n$. Then the KMP algorithm runs in average time*

$$T_{\text{KMP}}(u_1, u_2) = |\mathcal{L}_1| + |\mathcal{L}_2| + N_{\mathcal{L}_1 \bowtie \mathcal{L}_2} + |\mathcal{L}|$$

where

$$|\mathcal{L}_i| = \frac{n!}{(n - u_i)!}, \quad N_{\mathcal{L}_1 \bowtie \mathcal{L}_2} = \frac{(n!)^2 q^{\ell(n-r-u_1-u_2)}}{(n - u_1)!(n - u_2)!},$$

Algorithm 2 KMP(H, V, E)

Parameters: $u_1, u_2 \in \mathbb{N}^*$ s.t. $n - r + 1 \leq u_1 + u_2 \leq n$

Input: $H \in \mathbb{F}_q^{r \times n}$, $V \in \mathbb{F}_q^{\ell \times n}$, $E \in \mathbb{F}_q^{\ell \times r}$

Output: $\pi \in \text{Sym}(n)$ such that $\pi(V)H^\top = E$

```

1:  $\tilde{r} \leftarrow u_1 + u_2 - (n - r)$ 
2:  $H', M \leftarrow \text{RREF}_{\{n-r+1, \dots, n\}}^*(H)$ ,  $[U \ I_r] \leftarrow H'$ 
3:  $\bar{H} \leftarrow H'_{\{1, \dots, \tilde{r}\}}^{\{1, \dots, \tilde{r}\}}$ ,  $M' \leftarrow M_{\{1, \dots, \tilde{r}\}}$ 
4:  $\bar{E} \leftarrow EM'^\top$ ,  $\bar{H}_1 \leftarrow \bar{H}_{\{1, \dots, u_1\}}$ ,  $\bar{H}_2 \leftarrow \bar{H}_{\{u_1+1, \dots, u_1+u_2\}}$ 
5:  $\mathcal{L}_1 \leftarrow \{(X, X\bar{H}_1^\top) \mid X \in \mathcal{S}_{u_1}(V)\}$ ,
6:  $\mathcal{L}_2 \leftarrow \{(X, \bar{E} - Y\bar{H}_2^\top) \mid Y \in \mathcal{S}_{u_2}(V)\}$ 
7:  $\mathcal{L} \leftarrow \mathcal{L}_1 \bowtie \mathcal{L}_2$ 
8: for  $\tilde{V}_1 \in \mathcal{L}$  do
9:    $\tilde{V}_2 \leftarrow EM^\top - \tilde{V}_1_{\{1, \dots, n-r\}}U^\top$ 
10:  if  $[\tilde{V}_1 \ \tilde{V}_2] \in \mathcal{S}_n(V)$  then
11:    Compute  $\pi \in \text{Sym}(n)$  s.t.  $[\tilde{V}_1 \ \tilde{V}_2] = \pi(V)$ 
12:    Return:  $\pi$ 

```

$$|\mathcal{L}| = \frac{n!}{(n - u_1 - u_2)!} q^{\ell(n-r-u_1-u_2)}.$$

Remark 4. We note that $N_{\mathcal{L}_1 \bowtie \mathcal{L}_2}$ is the estimated number of collisions between $\mathcal{L}_1$ and $\mathcal{L}_2$. Indeed we are searching for collisions between $\ell \times \tilde{r}$ matrices, so the average number of collisions can be estimated by the number of possible pairs (given by $|\mathcal{L}_1| \cdot |\mathcal{L}_2|$) multiplied by the probability of a pair to collide, that is $\frac{1}{q^{\tilde{r}\ell}}$. Since $|\mathcal{L}| \leq N_{\mathcal{L}_1 \bowtie \mathcal{L}_2}$ by construction (here the inequality is considered between the estimates), the complexity of the algorithm is dominated by $\max\{|\mathcal{L}_1|, |\mathcal{L}_2|, N_{\mathcal{L}_1 \bowtie \mathcal{L}_2}\}$ and the algorithm is optimized when $|\mathcal{L}_1| \approx |\mathcal{L}_2| \approx N_{\mathcal{L}_1 \bowtie \mathcal{L}_2}$, which implies that either $u_1 = u_2$ if $u_1 + u_2$ is even or $u_1 = u_2 \pm 1$ otherwise. For the sake of simplicity, let us consider from now on $u_1 = u_2 = u$ so that $|\mathcal{L}_1| = |\mathcal{L}_2| = \frac{n!}{(n-u)!}$. Then to optimize the algorithm it is necessary to require that $N_{\mathcal{L}_1 \bowtie \mathcal{L}_2} \leq \frac{n!}{(n-u)!}$, which is equivalent to

$$\frac{n!}{(n-u)!} \leq q^{\ell(2u+r-n)}.$$

Hence, u is chosen as the smallest integer which satisfies the above inequality.

Notice that increasing q allows us to choose a smaller u , which speeds up the algorithm; therefore, the hardest PKP instances are the ones in which q is chosen adaptively as the smallest prime or prime power that guarantees the uniqueness of the solution, that is, the smallest q , that satisfies Condition iv) in Section 2.1. Moreover, notice that choosing q as small as possible has other advantages when considering applications of PKP to cryptosystems, such as speeding up the encryption/decryption or sign/verify and reducing the bit-size of keys or witnesses

in ZK-proofs (see [24]).

Finally, in [24] it was also shown that the complexity of KMP does not asymptotically depend on the value of ℓ and, even in the finite regime, increasing ℓ does not significantly change the running time (this will be examined in Section 5).

3.2 SBC algorithm

The SBC algorithm works similarly to KMP, but with an additional *filtering step*. Instead of building the lists as in Equation (8), here the idea is to choose the elements of the second list from a smaller set of matrices (that is why the algorithm can be considered as a version of KMP with an additional filtering step). Suppose that we want to solve an (n, r, q, ℓ) -IPKP instance $\pi(\mathbf{V})\mathbf{H}^\top = \mathbf{E}$. As usual, we denote $\pi(\mathbf{V})$ by $\tilde{\mathbf{V}}$. We now provide a description of the algorithm divided into 3 steps.

STEP 1: filtering step. The first step of the algorithm is to find a subcode of the linear code generated by $\mathbf{H}$ with support size w and dimension d , where w and d will be two parameters taken as input by the algorithm (actually, instead of w , the algorithm will take in input $w_1, w_2 \in \mathbb{N}^*$ with $w_1 + w_2 = w$ since, again, a *meet-in-the-middle* approach is used). Notice that it is not possible to find such subcode for every $w \in [n]$ and for every $d \in [r]$, therefore a necessary condition for w, d to be valid parameters is that $N_{n,r}(w, d) > 1$. Assuming one subcode with the desired properties has been found, let $\mathbf{H}^* \in \mathbb{F}_q^{d \times n}$ be its generator matrix. We have that $\mathbf{H}^* = \mathbf{S}\mathbf{H}$ for some $\mathbf{S} \in \text{GL}_{d,r}(\mathbb{F}_q)$. We will use the relation between $\mathbf{H}^*$ and $\mathbf{E}^* = \mathbf{E}\mathbf{S}^\top$ to filter the candidates for $\tilde{\mathbf{V}}_L$ where $L \subseteq [n]$ has size w . As in KMP, the idea is to guess $n - r + \tilde{r}$ columns with $\tilde{r} \geq 1$; however, this time $\tilde{r}$ is not fixed as in KMP, but it is a parameter itself that is taken as input by the algorithm with the condition $d \leq \tilde{r} \leq r$.

Remark 5. We now show why in the algorithm it is useful to consider another auxiliary permutation of the columns of $\mathbf{V}$ and why the condition $\tilde{r} \geq d$ arises. As in [24], we consider another permutation $\sigma \in \text{Sym}(n)$ such that $\sigma(\mathbf{H}^*)$ has its w non-null columns in positions indexed by $J = \{n - r + \tilde{r} - w + 1, \dots, n - r + \tilde{r}\}$. This choice simplifies the description of the algorithm because, at some point, $\sigma(\mathbf{H})$ will be brought into *Row Reduced Echelon Form* with respect to $\{n - r + 1, \dots, n\}$, and this arrangement of σ ensures that this can be done (i.e. it guarantees that $\sigma(\mathbf{H})_{\{n-r+1, \dots, n\}}$ is non-singular). To see this, assume, for example, that σ moves the non-null columns of $\mathbf{H}^*$ to the first w positions. Now let $\mathbf{S} \in \text{GL}_{d,r}(\mathbb{F}_q)$ be such that $\mathbf{S}\sigma(\mathbf{H}) = \sigma(\mathbf{H}^*)$ (notice that this is the same matrix $\mathbf{S}$ defined before, so $\mathbf{H}^* = \mathbf{S}\mathbf{H}$), and define $\mathbf{N} = \begin{bmatrix} \mathbf{S}' \\ \mathbf{S} \end{bmatrix}$, where $\mathbf{S}' \in \mathbb{F}_q^{(r-d) \times r}$ is chosen so that $\mathbf{N}$ is non singular. Then $\mathbf{N}\sigma(\mathbf{H})$ has, in its bottom-rightmost part, a $d \times (n - w)$ null matrix. To have that $\mathbf{N}\sigma(\mathbf{H})_{\{n-r+1, \dots, n\}}$ is non-singular (which means that it is possible to compute $\text{RREF}_{\{n-r+1, \dots, n\}}(\sigma(\mathbf{H}))$), since $\mathbf{N}\sigma(\mathbf{H})$ and $\sigma(\mathbf{H})$ share the same *Row Reduced Echelon Form*, it is necessary that it does not have linearly

dependent rows. Since the last d of them have zeros in $n - w$ entries, this implies that $r - (n - w) \geq d$. This is equivalent to $w \geq n - r + d$. However, in this case, the *filtering step* would be as expensive as the KMP algorithm itself, since we are guessing $w \geq n - r + d \geq n - r + 1$ columns and in KMP we were guessing $v \geq n - r + 1$ columns. Instead, if $\sigma \in \text{Sym}(n)$ moves the non-null columns of $\mathbf{H}^*$ in positions $\{n - r + \tilde{r} - w + 1, \dots, n - r + \tilde{r}\}$, we have that $\mathbf{N}\sigma(\mathbf{H})$ has a $d \times (r - \tilde{r})$ null matrix in its bottom-rightmost matrix. Thus, for $\mathbf{N}\sigma(\mathbf{H})_{\{n-r+1, \dots, n\}}$ to be non-singular, it is necessary that $r - (r - \tilde{r}) \geq d$, which is set by hypothesis.

Consider now $w_1, w_2 \in \mathbb{N}^*$ with $w_1 + w_2 = w$ and set $J_1 = \{n - r + \tilde{r} - w + 1, \dots, n - r + \tilde{r} - w_2\}$ and $J_2 = \{n - r + \tilde{r} - w_2 + 1, \dots, n - r + \tilde{r}\}$ (notice that $|J_1| = w - w_2 = w_1$ and $|J_2| = w_2$). Let $\mathbf{H}_1^* = \sigma(\mathbf{H}^*)_{J_1}$ and $\mathbf{H}_2^* = \sigma(\mathbf{H}^*)_{J_2}$. Then $\sigma(\mathbf{H}^*)$ can be visualized as follows.

$$\sigma(\mathbf{H}^*) = d \left\| \begin{array}{c|c|c|c} n-r+\tilde{r}-w & w_1 & w_2 & \\ \hline \mathbf{0} & \mathbf{H}_1^* & \mathbf{H}_2^* & \mathbf{0} \end{array} \right\|$$

We further define $J = J_1 \cup J_2 = \{n - r + \tilde{r} - w + 1, \dots, n - r + \tilde{r}\}$ and $\tilde{\mathbf{H}} = \sigma(\mathbf{H}^*)_J = [\mathbf{H}_1^* \ \mathbf{H}_2^*]$. Then the algorithm proceeds by searching for candidates for $\sigma(\tilde{\mathbf{V}})_J$ in the list

$$\mathcal{K} = \{\mathbf{X} \in \mathcal{S}_w(\mathbf{V}) \mid \mathbf{X}\tilde{\mathbf{H}}^\top = \mathbf{E}^*\}. \quad (9)$$

We will show now that $\sigma(\tilde{\mathbf{V}})_J \in \mathcal{K}$. As $\tilde{\mathbf{V}}\mathbf{H}^\top = \mathbf{E}$, then it is easy to see that

$$\mathbf{E}\mathbf{A}^\top = \sigma(\tilde{\mathbf{V}})(\mathbf{A}\sigma(\mathbf{H}))^\top, \quad \forall \mathbf{A} \in \mathbb{F}_q^{r' \times r}, \forall r' \leq r, \forall \sigma \in \text{Sym}(n). \quad (10)$$

Since $\mathbf{H}^* = \mathbf{S}\mathbf{H}$ we have that $\mathbf{E}^* = \mathbf{E}\mathbf{S}^\top = \sigma(\tilde{\mathbf{V}})\sigma(\mathbf{H}^*)^\top$ and $\sigma(\mathbf{H}^*)$ has the structure showed in the figure above. Therefore, if we call $J' = \{1, \dots, n - r + \tilde{r} - w\}$ and $J'' = \{n - r + \tilde{r} + 1, \dots, n\}$, we have

$$\mathbf{E}^* = \sigma(\tilde{\mathbf{V}})\sigma(\mathbf{H}^*)^\top = [\sigma(\tilde{\mathbf{V}})_{J'} \ \sigma(\tilde{\mathbf{V}})_J \ \sigma(\tilde{\mathbf{V}})_{J''}] \begin{bmatrix} \mathbf{0} \\ \tilde{\mathbf{H}}^\top \\ \mathbf{0} \end{bmatrix} = \sigma(\tilde{\mathbf{V}})_J \tilde{\mathbf{H}}^\top.$$

To build the list $\mathcal{K}$, a *meet-in-the-middle* approach is used, so instead of building it like in Equation (9), we build $\mathcal{K}$ as $\mathcal{K}_1 \bowtie \mathcal{K}_2$, with

$$\begin{aligned} \mathcal{K}_1 &= \{(\mathbf{X}_1, \mathbf{X}_1 \mathbf{H}_1^{*\top}) \mid \mathbf{X}_1 \in \mathcal{S}_{w_1}(\mathbf{V})\}, \\ \mathcal{K}_2 &= \{(\mathbf{X}_2, \mathbf{E}^* - \mathbf{X}_2 \mathbf{H}_2^{*\top}) \mid \mathbf{X}_2 \in \mathcal{S}_{w_2}(\mathbf{V})\}. \end{aligned}$$

STEP 2: extra guessing. From now on, we assume that $\{n - r + 1, \dots, n\}$ is an information set for $\sigma(\mathbf{H})$, but the reasoning works for every information set.

After the *filtering step*, we obtain a list of candidates for $\sigma(\tilde{\mathbf{V}})_J$, with $|J| = w$. Because of the considerations made in Remark 5, if $w \geq n - r + d$ we know enough columns to reconstruct the whole solution by exploiting the kernel dimension (as in Lines 4-7 in Algorithm 1). As observed in the same remark, the interesting values of w are the ones for which $w < n - r + d$ (otherwise the complexity of the algorithm would be essentially the same as KMP). If this is the case, we need to guess $n - r - w + \tilde{r}$ more columns. Notice that guessing $n - r - w + d$ more columns would be sufficient, however since this guessing is done by using a *meet-in-the-middle* approach, the algorithm might be optimized when guessing more columns.

The algorithm proceeds by computing $\mathbf{H}'$, $\mathbf{M} = \text{RREF}^*_{\{n-r+1, \dots, n\}}(\sigma(\mathbf{H}))$. Now we call $\hat{\mathbf{H}} = \mathbf{H}'^{\{d+1, \dots, \tilde{r}\}}$, $\hat{\mathbf{E}} = (\mathbf{E}\mathbf{M}^\top)_{\{d+1, \dots, \tilde{r}\}}$, $\hat{\mathbf{H}}_1 = \hat{\mathbf{H}}_{J'}$ and $\hat{\mathbf{H}}_2 = \hat{\mathbf{H}}_J$ and build the lists

$$\begin{aligned}\mathcal{L}_1 &= \left\{ (\mathbf{Y}_1, \mathbf{Y}_1 \hat{\mathbf{H}}_1^\top) \mid \mathbf{Y}_1 \in \mathcal{S}_{n-r+\tilde{r}-w}(\mathbf{V}) \right\}, \\ \mathcal{L}_2 &= \left\{ (\mathbf{Y}_2, \hat{\mathbf{E}} - \mathbf{Y}_2 \hat{\mathbf{H}}_2^\top) \mid \mathbf{Y}_2 \in \mathcal{K} \right\}, \\ \mathcal{L} &= \mathcal{L}_1 \bowtie \mathcal{L}_2.\end{aligned}$$

It is easy to see that $\sigma(\tilde{\mathbf{V}})_{J' \cup J} \in \mathcal{L}$. To visualize this, we refer to Figure 2.

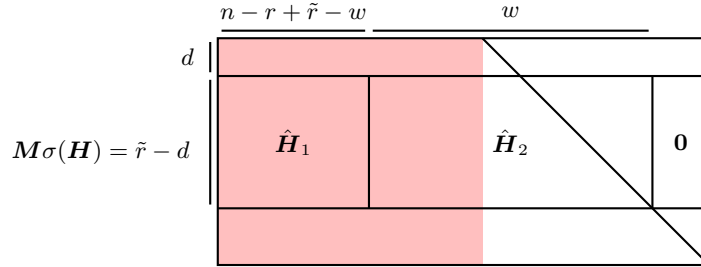


Fig. 2: Matrices employed in the SBC algorithm. The pink area is the matrix $\mathbf{U}$ defined in Algorithm 3.

Indeed, from Equation (10) we have that $\mathbf{E}\mathbf{M}^\top = \sigma(\tilde{\mathbf{V}})(\mathbf{M}\sigma(\mathbf{H}))^\top$. This implies

$$(\mathbf{E}\mathbf{M}^\top)_{\{d+1, \dots, \tilde{r}\}} = \left(\sigma(\tilde{\mathbf{V}})(\mathbf{M}\sigma(\mathbf{H}))^\top \right)_{\{d+1, \dots, \tilde{r}\}} = \sigma(\tilde{\mathbf{V}}) \left((\mathbf{M}\sigma(\mathbf{H}))^\top \right)_{\{d+1, \dots, \tilde{r}\}}.$$

Observing that selecting a set of columns of the transpose is the same as selecting that set of rows of the non-transpose, we obtain that:

$$\hat{\mathbf{E}} = \sigma(\tilde{\mathbf{V}})\hat{\mathbf{H}}^\top = [\sigma(\tilde{\mathbf{V}})_{J'} \sigma(\tilde{\mathbf{V}})_J \sigma(\tilde{\mathbf{V}})_{J''}] \begin{bmatrix} \hat{\mathbf{H}}_1^\top \\ \hat{\mathbf{H}}_2^\top \\ \mathbf{0} \end{bmatrix} = [\sigma(\tilde{\mathbf{V}})_{J'} \sigma(\tilde{\mathbf{V}})_J] \begin{bmatrix} \hat{\mathbf{H}}_1^\top \\ \hat{\mathbf{H}}_2^\top \end{bmatrix}.$$

Notice that the last columns $r - \tilde{r}$ of $\hat{\mathbf{H}}$ are zero because they are obtained by taking a null submatrix of $\mathbf{I}_r$. From the first step of the algorithm we have that $\sigma(\tilde{\mathbf{V}})_J \in \mathcal{K}$, so $\sigma(\tilde{\mathbf{V}})_{J' \cup J} \in \mathcal{L}$.

STEP 3: reconstruct the solution. As said before, at the end of **STEP 2** we know enough columns of $\sigma(\pi(\mathbf{V}))$ to reconstruct the solution working as in Section 2.2 and in the last part of KMP (that is, by exploiting the kernel dimension).

Remark 6. In Algorithm 3, in **STEP 3**, the solution is reconstructed by considering the last $r - \tilde{r}$ rows of $\mathbf{U}$ and consequently the last $r - \tilde{r}$ columns of $\mathbf{E}\mathbf{M}^\top$. This is done because we are looking for $\mathbf{X}'$ such that $[\mathbf{X} \ \mathbf{X}'] \in \mathcal{S}_n(\mathbf{V})$ and $[\mathbf{X} \ \mathbf{X}'] \begin{bmatrix} \mathbf{U}^\top \\ \mathbf{I}_r \end{bmatrix} = \mathbf{E}\mathbf{M}^\top$. However, we can just check the equality between the last $r - \tilde{r}$ columns of these matrices, because the equality between the first $n - r + \tilde{r}$ columns has already been checked in **STEP 2** (a similar reasoning also applies when discarding the first d columns after **STEP 1**).

Complexity of SBC. The next proposition provides the average running time of SBC.

Proposition 2 (Complexity of SBC [24, Prop. 9]). *Let $d, w_1, w_2, \tilde{r} \in \mathbb{N}^*$ be such that $w = w_1 + w_2 \leq n$, $d \leq \tilde{r} \leq r$ and $N_{n,r}(w, d) > 1$. Then Algorithm 3 runs in average time*

$$T_{\text{SBC}}(n, r, q, \ell, w_1, w_2, d, \tilde{r}) = T_{\text{ISD}}^{(d)}(n, r, w) + T_{\mathcal{K}} + T_{\mathcal{L}} + |\mathcal{L}| \quad (11)$$

where $T_{\text{ISD}}^{(d)}(n, r, w)$ is defined as in Equation (3) and

$$T_{\mathcal{K}} = \frac{n!}{(n - w_1)!} + \frac{n!}{(n - w_2)!} + \frac{(n!)^2 q^{-d\ell}}{(n - w_1)!(n - w_2)!},$$

$$T_{\mathcal{L}} = \frac{n!}{(r + w - \tilde{r})!} + |\mathcal{K}| + \frac{n!|\mathcal{K}|q^{-\ell(\tilde{r}-d)}}{(r + w - \tilde{r})!}, \quad (12)$$

$$|\mathcal{L}| = \frac{(n - w)!|\mathcal{K}|q^{-\ell(\tilde{r}-d)}}{(r - \tilde{r})!}, |\mathcal{K}| = \max \left\{ \frac{n!q^{-d\ell}}{(n - w)!}, 1 \right\}. \quad (13)$$

Remark 7. In Proposition 2, $\tilde{r}$ is allowed to be equal to d . In this case, in **STEP 2**, one would build matrices with zero rows/columns. This is, in some sense, taken into account in Proposition 2, but in this case the complexity of the algorithm is slightly overestimated. If $\tilde{r} = d$, then for every candidate $\mathbf{X} \in \mathcal{K}$, one should try to reconstruct the solution for every possible way of choosing $n - r + \tilde{r} - w$ columns of $\mathbf{V}$ (without checking for collisions as in **STEP 2**) between the $n - w$ columns that do not appear in $\mathbf{X}$. This would lead to a slightly smaller complexity than the one showed in Proposition 2, because the complexity given by Equation (11) assumes that for every candidate the extra

Algorithm 3 SBC(H, V, E)

Parameters: $w_1, w_2, \tilde{r}, d \in \mathbb{N}$ with $w = w_1 + w_2 \leq n$, $d \leq \tilde{r} \leq r$ and $N_{n,r}(w, d) > 1$

Input: $H \in \text{GL}_{r,n}(\mathbb{F}_q)$, $V \in \text{GL}_{\ell,n}(\mathbb{F}_q)$, $E \in \mathbb{F}_q^{\ell \times r}$

Output: $\pi \in \text{Sym}(n)$ such that $\pi(V)H^\top = E$

STEP 1: filtering step

- 1: Use ISD to find a subcode of the code generated by H with dimension d and support size $w = w_1 + w_2$. Call $\mathcal{C}$ this subcode, H^* its generator matrix and $S \in \text{GL}_{d,r}(\mathbb{F}_q)$ the matrix s.t. $H^* = SH$.
- 2: Compute $\sigma \in \text{Sym}(n)$ such that $\text{Supp}(\sigma(\mathcal{C})) = \{n-r+\tilde{r}-w+1, \dots, n-r+\tilde{r}\}$
- 3: $J_1 \leftarrow \{n-r+\tilde{r}-w+1, \dots, n-r+\tilde{r}-w_2\}$, $J_2 \leftarrow \{n-r+\tilde{r}-w_2+1, \dots, n-r+\tilde{r}\}$
- 4: $E^* \leftarrow ES^\top$, $H_1^* \leftarrow \sigma(H^*)_{J_1}$, $H_2^* \leftarrow \sigma(H^*)_{J_2}$
- 5: $\mathcal{K}_1 \leftarrow \{(X_1, X_1 H_1^{*\top}) \mid X_1 \in \mathcal{S}_{w_1}(V)\}$,
- 6: $\mathcal{K}_2 \leftarrow \{(X_2, E^* - X_2 H_2^{*\top}) \mid X_2 \in \mathcal{S}_{w_2}(V)\}$
- 7: $\mathcal{K} \leftarrow \mathcal{K}_1 \bowtie \mathcal{K}_2$

STEP 2: extra guessing

- 8: $H', M \leftarrow \text{RREF}_{\{n-r+1, \dots, n\}}^*(\sigma(H))$, $[U \ I_r] \leftarrow H'$
- 9: $\hat{H} \leftarrow H'^{\{d+1, \dots, \tilde{r}\}}$, $\hat{E} \leftarrow (EM^\top)_{\{d+1, \dots, \tilde{r}\}}$
- 10: $J' \leftarrow \{1, \dots, n-r+\tilde{r}-w\}$, $J \leftarrow J_1 \cup J_2$, $\hat{H}_1 \leftarrow \hat{H}_{J'}$, $\hat{H}_2 \leftarrow \hat{H}_J$
- 11: $\mathcal{L}_1 = \{(Y_1, Y_1 \hat{H}_1^\top) \mid Y_1 \in \mathcal{S}_{n-r+\tilde{r}-w}(V)\}$
- 12: $\mathcal{L}_2 = \{(Y_2, \hat{E} - Y_2 \hat{H}_2^\top) \mid Y_2 \in \mathcal{K}\}$
- 13: $\mathcal{L} = \mathcal{L}_1 \bowtie \mathcal{L}_2$

STEP 3: reconstruct the solution

- 14: $\bar{U} \leftarrow U^{\{\tilde{r}+1, \dots, r\}}$, $\bar{E} \leftarrow (EM^\top)_{\{\tilde{r}+1, \dots, r\}}$
 - 15: **for** $X \in \mathcal{L}$ **do**
 - 16: $X' \leftarrow \bar{E} - X_{\{1, \dots, n-r\}} \bar{U}^\top$
 - 17: **if** $[X \ X'] \in \mathcal{S}_n(V)$ **then**
 - 18: Compute $\pi \in \text{Sym}(n)$ s.t. $\sigma^{-1}([X \ X']) = \pi(V)$
 - 19: **Return:** π
-

guessing is made between all the n columns of V instead of between the $n-w$ remaining ones. Therefore, the term $T_{\mathcal{L}} + |\mathcal{L}|$ in Equation (11) should be replaced by $\frac{(n-w)!|\mathcal{L}|}{(r-\tilde{r})!}$. We point out that, based on our analysis, this does not lead to significant changes in the running time, but we report it for completeness.

4 New Algorithmic Improvements for Solving PKP

In this section, we introduce several improvements to SBC. These improvements apply to KMP as well, but since SBC outperforms KMP in the finite regime (see [24, Sec. 6.2]), that is of primary interest in this work, we focus on SBC. We introduce variants both for the *filtering step* (**STEP 1** in Algorithm 3) and

for the *extra guessing* (**STEP 2** in Algorithm 3). Then, we outline our refined new algorithm which chooses the most convenient variant of these steps for each choice of parameters.

In the following, we consider an (n, r, q, ℓ) -IPKP instance $\pi(\mathbf{V})\mathbf{H}^\top = \mathbf{E}$ with $\text{rank}(\mathbf{E}) = 1$ and in which the last row of $\mathbf{H}$ is a row of ones (this is the case of an IPKP-instance obtained by de-homogenizing a PKP instance) and use the same notation used in the previous section. Although the algorithm applies to arbitrary IPKP instances, we focus on instances with specific properties arising from the dehomogenization of a (homogeneous) PKP instance. Such IPKP instances cannot be treated as random in the complexity analysis, as dehomogenization introduces equations that hold for every permutation, which would otherwise lead to an underestimation of the number of solutions and hence of the complexity (see Remark 14).

4.1 Variant of STEP 2: Using the dual to guess more columns

We begin introducing our proposed variant of **STEP 2** of SBC because this modification applies also to our variant of **STEP 1** that is presented in Section 4.2. In a nutshell, the variant exploits the RREF of $\mathbf{V}$ instead of the one of $\mathbf{H}$ by considering the dual version of the problem instance (see Section 2.3).

Recall that **STEP 2** starts with a list $\mathcal{K}$ containing all the candidates for $\sigma(\tilde{\mathbf{V}})_J$ (with $J = \{n - r + \tilde{r} - w + 1, \dots, n - r + \tilde{r}\}$). By inverting σ , for each candidate $\mathbf{X}$ for $\sigma(\tilde{\mathbf{V}})_J$ (corresponding to a candidate permutation $\pi \in \text{Sym}(n)$), it is possible to determine $i_1, \dots, i_w$ s.t. $\pi(i_1) = j_1, \dots, \pi(i_w) = j_w$ with $\sigma(j_1) = n - r + \tilde{r} - w + 1, \dots, \sigma(j_w) = n - r + \tilde{r}$ and so we know $\pi^{-1}(j_1) = i_1, \dots, \pi^{-1}(j_w) = i_w$. In the following, we call $\text{InvCand}(\mathbf{X})$ the algorithm that takes in input a candidate $\mathbf{X}$ for $\sigma(\tilde{\mathbf{V}})_J$ and returns both the set $I_{\mathbf{X}} = \{i_1, \dots, i_w\}$ and the corresponding candidate for $\pi^{-1}(\mathbf{H})_{I_{\mathbf{X}}}$. Notice that this candidate is unique only if $\mathbf{V}$ has no repeated columns, we clarify this in the next section but for now we assume this is the case.

Consider now the dual version of the problem

$$\pi^{-1}(\mathbf{H})\mathbf{V}^\top = \mathbf{E}^\top.$$

In this setting, for every $\mathbf{X} \in \mathcal{K}$, we know a candidate for $\pi^{-1}(\mathbf{H})_{I_{\mathbf{X}}}$, where in general the set $I_{\mathbf{X}}$ is different for every candidate $\mathbf{X}$. For this reason, it is not possible to build the list $\mathcal{L}$ with a *meet-in-the-middle* approach since for every candidate we need to compute the RREF w.r.t. a (possibly) different information set. In the original approach, we needed to guess the action of the permutation on $n - r - w + \tilde{r}$ more columns to have enough information to reconstruct the solution. Here, if we extend $\mathbf{V}$, we need to guess $n - (\ell + 1) - w + \tilde{r}$ more columns of $\pi^{-1}(\mathbf{H})$. Thus, since we cannot use the *meet-in-the-middle* approach, for every element in $\mathcal{K}$ we try all the possible ways of choosing $n - (\ell + 1) - w + \tilde{r}$ columns from the $n - w$ remaining ones.

Another difference with the original **STEP 2** in Algorithm 3 is that here we consider any $0 \leq \tilde{r} \leq \ell + 1$ instead of requiring $\tilde{r} \geq d$. The latter was required

because of the considerations made in Remark 5 which are no longer valid in our case. In particular, the constraint was to avoid having a sub-matrix with zero columns in the positions indexed by the set used to compute the RREF, but since now we are computing the RREF of $\mathbf{V}$ instead of $\mathbf{H}$, no null sub-matrices are involved.

On the probability of failure. If we denote with $\mathbf{V}^{\text{ext}}$ the extended $\mathbf{V}$, we consider the probability of successfully computing $\text{RREF}_{I'_{\mathbf{X}}}(\mathbf{V}^{\text{ext}})$ w.r.t. a fixed set $I'_{\mathbf{X}} \subseteq [n]$ with $|I'_{\mathbf{X}}| = \ell + 1$ as the probability that a random $(\ell + 1) \times (\ell + 1)$ matrix is non-singular. However, we do not need to compute the RREF w.r.t. one fixed subset of indexes, as we usually have multiple possible choices. Indeed, we just need to compute the RREF w.r.t. a set $I'_{\mathbf{X}}$ that intersects the set $I_{\mathbf{X}}$ containing the indexes of the w known columns of $\pi^{-1}(\mathbf{H})$ in a set of cardinality $\tilde{r}$. This permits to use as much known columns as possible to reconstruct the solution. Since the possible choices of such $I'_{\mathbf{X}}$ are many, we can safely assume that the probability that every choice is not an information set for $\mathbf{V}^{\text{ext}}$ is negligible. We call $\text{RecoverInfSet}(\mathbf{X})$ the procedure that takes in input a candidate $\mathbf{X}$ for $\sigma(\tilde{\mathbf{V}})_J$ and returns a “valid” information set in the sense we just explained.

By joining the above variant of **STEP 2** with the corresponding adaptation of **STEP 3**, we obtain Algorithm 4. Note that it is not necessary (more precisely, it is not useful) to introduce the auxiliary permutation σ , since it is inverted before having played any role. Nevertheless, we still consider it in order to be consistent with the the original **STEP 2** in which σ is needed.

Remark 8. Notice that in Line 4 of Algorithm 4, we introduce an auxiliary permutation μ to simplify the description of the algorithm. This permutation is not required for the algorithm to function, but is introduced solely for clarity of exposition. Moreover, such a permutation exists because, by construction, $|\{i_1, \dots, i_w\} \cap I'_{\mathbf{X}}| = \tilde{r}$.

Remark 9. Instead of taking $\mathbf{V}$ and $\mathbf{E}$ as inputs, Algorithm 4 takes the extended matrices, which we call $\mathbf{V}^{\text{ext}}$ and $\mathbf{E}^{\text{ext}}$ respectively. We explicitly build these matrices, for example, at Line 15 of Algorithm 7. Another observation is that Algorithm 4 could, in principle, terminate as soon as a solution is found. Nevertheless, we intentionally make it not terminate to account for instances with multiple solutions, which we encounter in the following section.

Proposition 3. *Let $\mathbf{H} \in \text{GL}_{r,n}(\mathbb{F}_q)$, $\mathbf{V} \in \text{GL}_{\ell+1,n}(\mathbb{F}_q)$ be matrices with no repeated columns and $\mathbf{E} \in \mathbb{F}_q^{r \times (\ell+1)}$. Let $w, d \in \mathbb{N}^*$, $\tilde{r} \in \mathbb{N}$ s.t. $w \leq n$, $0 \leq \tilde{r} \leq \ell + 1$ and $N_{n,r}(w, d) > 1$. Let $\tilde{\mathbf{V}} = \pi(\mathbf{V})$ and let $\mathcal{K}$ be the list containing the candidates for $\sigma(\tilde{\mathbf{V}})_J$, with $J \subseteq [n]$ and $|J| = w$. Then the running time of Algorithm 4 is given by*

$$\bar{T}_{\mathcal{L}}(n, \ell, w, \tilde{r}, |\mathcal{K}|) = \begin{cases} |\mathcal{K}|^{\frac{(n-w)!}{(\ell+1-\tilde{r})!}} & \text{if } \ell + 1 - \tilde{r} \leq n - w, \\ |\mathcal{K}| & \text{otherwise.} \end{cases} \quad (14)$$

Algorithm 4 DualGuessSolve($H, V^{\text{ext}}, E^{\text{ext}}, \mathcal{K}$)

Parameters: $w, \tilde{r}, d \in \mathbb{N}^*$ with $0 \leq \tilde{r} \leq \ell + 1$ and $N_{n,r}(w, d) > 1$

Input: $H \in \text{GL}_{r,n}(\mathbb{F}_q)$, $V^{\text{ext}} \in \text{GL}_{\ell+1,n}(\mathbb{F}_q)$, $E^{\text{ext}} \in \mathbb{F}_q^{r \times (\ell+1)}$, $\mathcal{K}$ containing all candidates for $\sigma(\tilde{V})_J$

Output: A list $\mathcal{L}$ containing $\tilde{H} \in \mathcal{S}_n(H)$ such that $\tilde{H}(V^{\text{ext}})^\top = E^{\text{ext}}$

- 1: Initialize an empty list $\mathcal{L}$
- 2: **for** $X \in \mathcal{K}$ **do**
- 3: $\{i_1, \dots, i_w\}, Y \leftarrow \text{InvCand}(X), I'_X \leftarrow \text{RecoverInfSet}(X)$
- 4: Let $\mu \in \text{Sym}(n)$ s.t. $\mu(I'_X) = \{n - \ell, \dots, n\}$ and

$$\mu(i_1) = n - (\ell + 1) - w + \tilde{r} + 1, \dots, \mu(i_w) = n - (\ell + 1) + \tilde{r}$$

- 5: $V', M \leftarrow \text{RREF}^*_{\{n-\ell, \dots, n\}}(\mu(V^{\text{ext}})), U \leftarrow V'_{\{1, \dots, n-(\ell+1)\}}$
 - 6: $E' \leftarrow (E^{\text{ext}} M^\top)_{\{1, \dots, \tilde{r}\}}$
 - 7: **for** $Z \in \mathcal{S}_{n-(\ell+1)-w+\tilde{r}}(H_{\neg_n \{i_1, \dots, i_w\}})$ **do**
 - 8: $\tilde{V} \leftarrow V'^{\{1, \dots, \tilde{r}\}}_{\{1, \dots, n-(\ell+1)+\tilde{r}\}}$
 - 9: $W \leftarrow [Z \ Y]$
 - 10: **if** $W \tilde{V}^\top = E'$ **then**
 - 11: $W' \leftarrow (E^{\text{ext}} M^\top) - W_{\{1, \dots, n-(\ell+1)\}} U^\top$
 - 12: **if** $[W_{\{1, \dots, n-(\ell+1)\}} \ W'] \in \mathcal{S}_n(H)$ **then**
 - 13: $\mathcal{L} \leftarrow \mathcal{L} \cup \{ \mu^{-1}([W_{\{1, \dots, n-(\ell+1)\}} \ W']) \}$
 - 14: **Return:** $\mathcal{L}$
-

Proof. We first consider the case $\ell + 1 - \tilde{r} \leq n - w$. The cost of Line 3, because of the considerations above, can be assumed to be absorbed by the $\mathcal{O}$ -notation. Therefore the overall complexity is given by $|\mathcal{K}| \cdot |\mathcal{S}_{n-(\ell+1)-w+\tilde{r}}(H_{\neg_n \{i_1, \dots, i_w\}})|$. Finally, if $\ell + 1 - \tilde{r} > n - w$, then, after the *filtering step*, we know enough columns to directly reconstruct the solution (which means that in the for loop at Line 7 it is not needed to compute W since we already know it). $\square$

Remark 10. In Proposition 3 we allow $\tilde{r} = 0$, even if this means considering matrices with zero rows/columns. In this case, analogously to Remark 7, we try to reconstruct the solution directly from every candidate in $\mathcal{K}$. This observation does not affect the complexity of the algorithm given by Equation (14).

Remark 11. Note that our variant of **STEP 2** is not always an improvement w.r.t. the original version. Depending on the parameters, it might be more convenient to choose one version or the other. The two variants are compared in Table 1. Note that the last row in the table, corresponding to the dimensions of the colliding matrices, does not affect directly the running time of the algorithm as it is proposed now. However, as shown in this section, it affects the number of expected solutions. This affects the running time of our newly proposed Algorithm 7, see Lemma 2.

	Original STEP 2	Our variant
Amount of extra guesses	$n - r + \tilde{r} - w$	$n - (\ell + 1) + \tilde{r} - w$
Constraint on $\tilde{r}$	$d \leq \tilde{r} \leq r$	$0 \leq \tilde{r} \leq \ell + 1$
<i>Meet-in-the-middle</i>	✓	×
Dimensions of colliding matrices	$\ell \times (\tilde{r} - d)$	$\tilde{r} \times (r - 1)$

Table 1: Comparison of the two variants. Red cells indicate the variant that results in higher complexity for each respective parameter.

4.2 Variant of STEP 1: Recursive Filtering

We introduce here a variant for the *filtering step* of SBC (**STEP 1** in Algorithm 3). The idea is to replace the *meet-in-the-middle* approach by solving multiple IPKP (sub)instances. Recall that the goal of the *filtering step* is to have a list $\mathcal{K} = \{\mathbf{X} \in \mathcal{S}_w(\mathbf{V}) \mid \mathbf{X}\bar{\mathbf{H}}^\top = \mathbf{E}^*\}$ containing candidates for $\sigma(\tilde{\mathbf{V}})_J$ (see Equation (9)). For every $W \subseteq [n]$, with $|W| = w$, we populate the list $\mathcal{K}$ with elements of the form $\tilde{\pi}(\mathbf{V}_W)$, for some $\tilde{\pi} \in \text{Sym}(w)$, such that

$$\tilde{\pi}(\mathbf{V}_W)\bar{\mathbf{H}}^\top = \mathbf{E}^*, \quad (15)$$

which is a (w, d, q, ℓ) -IPKP instance. In total, that are $\binom{n}{w}$ possible choices for W . Notice that, for simplicity, we abuse the terminology of Definition 2 and refer to IPKP instances also in cases they have no solution. This can occur when the chosen set W is not the one containing the columns of $\sigma(\tilde{\mathbf{V}})_J$.

We only consider IPKP instances in their dual form (see Equation (6)), as our complexity analysis shows this to be consistently the best choice. To increase the row dimension of the instances by one, we extend $\mathbf{V}_W$, which in the considered dual instances plays the role of the parity-check, by adding an additional row of ones analogously to Section 2.2. This means that $\mathcal{K}$ collects solutions of IPKP instances with parameters $(w, \ell + 1, q, d)$. We highlight that these subinstances might also have multiple solutions. In order to solve them, we use a slightly modified version of SBC.

Remark 12. To be precise, the subinstances to solve (after having extended $\mathbf{V}_W$) have parameters $(w, \text{rank}(\mathbf{V}_W) + 1, q, d)$. In the following, we assume that $\text{rank}(\mathbf{V}_W) = \min\{\ell, w - d + 1\}$. We motivate this in Appendix A.

Dealing with instances with multiple solutions. Let $\ell^* := \text{rank}(\mathbf{V}_W)$. In most cases, the subinstances under consideration have multiple solutions that need to be found all. If $\bar{\mathbf{H}}$ has no repeated columns, we expect multiple solutions to occur whenever condition iv) in Section 2.1 does not hold (that is, when $\frac{w!}{q^{\ell^*d}} \geq 1$). In this case, Algorithm 3 can be easily adapted to return all solutions: it suffices to output all valid permutations instead of terminating as soon as one is found (as in Lines 13–14 of Algorithm 4). We define SBC* (Algorithm 5)

and $\text{SBC}_{\text{dual}}^*$ (Algorithm 6) as the variants of SBC (Algorithm 3) that take this into account, performing respectively the original **STEP 2-3** of Algorithm 3 and Algorithm 4. Suppose we are solving the instance $\pi(\mathbf{V})\mathbf{H}^\top = \mathbf{E}$. Differently from Algorithm 3, Algorithm 5 and Algorithm 6 return lists containing, respectively, all $\tilde{\mathbf{V}} \in \mathcal{S}_n(\mathbf{V})$ such that $\tilde{\mathbf{V}}\mathbf{H}^\top = \mathbf{E}$, and all $\tilde{\mathbf{H}} \in \mathcal{S}_n(\mathbf{H})$ such that $\tilde{\mathbf{H}}(\mathbf{V}^{\text{ext}})^\top = \mathbf{E}^{\text{ext}}$, where $\mathbf{V}^{\text{ext}}$ is the matrix obtained by extending $\mathbf{V}$ with a row of ones and $\mathbf{E}^{\text{ext}}$ is the matrix obtained by extending $\mathbf{E}^\top$ accordingly. We explicit these algorithms in Algorithms 5 and 6.

Algorithm 5 $\text{SBC}^*(\mathbf{H}, \mathbf{V}, \mathbf{E})$

Parameters: $w_1, w_2, \tilde{r}, d \in \mathbb{N}$ with $w = w_1 + w_2 \leq n$, $d \leq \tilde{r} \leq r$ and

$$N_{n,r}(w, d) > 1$$

Input: $\mathbf{H} \in \text{GL}_{r,n}(\mathbb{F}_q)$, $\mathbf{V} \in \text{GL}_{\ell,n}(\mathbb{F}_q)$, $\mathbf{E} \in \mathbb{F}_q^{\ell \times r}$

Output: $\pi \in \text{Sym}(n)$ such that $\pi(\mathbf{V})\mathbf{H}^\top = \mathbf{E}$

- 1: Proceed as in Lines 1-14 of Algorithm 3
 - 2: Initialize an empty list $\mathcal{T}$
 - 3: **for** $\mathbf{X} \in \mathcal{L}$ **do**
 - 4: $\mathbf{X}' \leftarrow \bar{\mathbf{E}} - \mathbf{X}_{\{1, \dots, n-r\}} \bar{\mathbf{U}}^\top$
 - 5: **if** $[\mathbf{X} \ \mathbf{X}'] \in \mathcal{S}_n(\mathbf{V})$ **then**
 - 6: $\mathcal{T} \leftarrow \mathcal{T} \cup \{\sigma^{-1}([\mathbf{X} \ \mathbf{X}'])\}$
 - 7: **Return:** $\mathcal{T}$
-

Algorithm 6 $\text{SBC}_{\text{dual}}^*(\mathbf{H}, \mathbf{V}, \mathbf{E})$

Parameters: $w_1, w_2, \tilde{r}, d \in \mathbb{N}$ with $w = w_1 + w_2 \leq n$, $d \leq \tilde{r} \leq r$ and

$$N_{n,r}(w, d) > 1$$

Input: $\mathbf{H} \in \text{GL}_{r,n}(\mathbb{F}_q)$, $\mathbf{V} \in \text{GL}_{\ell,n}(\mathbb{F}_q)$, $\mathbf{E} \in \mathbb{F}_q^{\ell \times r}$

Output: $\pi \in \text{Sym}(n)$ such that $\pi(\mathbf{V})\mathbf{H}^\top = \mathbf{E}$

- 1: Proceed as in Lines 1-7 of Algorithm 3
 - 2: $\mathbf{V}^{\text{ext}} \leftarrow \begin{bmatrix} \mathbf{V} \\ (1, \dots, 1) \end{bmatrix}$, $\mathbf{E}^{\text{ext}} \leftarrow [\mathbf{E}^\top \sum_{i=1}^n \mathbf{h}_i]$
 - 3: **Return:** $\text{DualGuessSolve}(\mathbf{H}, \mathbf{V}^{\text{ext}}, \mathbf{E}^{\text{ext}}, \mathcal{K})$
-

Allowing matrices to have repeated columns. We must solve IPKP instances with parameters $(w, \ell^* + 1, q, d)$ obtained by considering the dual version of Equation (15), that is

$$\tilde{\pi}^{-1}(\tilde{\mathbf{H}})\mathbf{V}_W^\top = \mathbf{E}^{*\top}.$$

We recall that $\tilde{\mathbf{H}}$ is the matrix formed by the nonzero columns of the generator matrix of a random subcode of the code generated by $\mathbf{H}$, with dimension d and support size w (so $d < w$). We model $\tilde{\mathbf{H}}$ as a random element of $\text{GL}_{d,w}(\mathbb{F}_q)$

with no zero columns, meaning that there might be repeated columns. If this is the case, multiple solutions to the instance necessarily exist. Let us suppose, for example, that $\bar{\mathbf{H}}$ has 2 identical columns and all the others are distinct. If this happens and $\tilde{\pi}^{-1} \in \text{Sym}(w)$ is a solution, another solution $\tilde{\mu} \in \text{Sym}(w)$ can be found by swapping the images under $\tilde{\pi}^{-1}$ of the indexes corresponding to the repeating columns in $\bar{\mathbf{H}}$.

Definition 3. If $\mathbf{H} \in \mathbb{F}_q^{r \times n}$ is a matrix with no zero columns, we define $\text{COL}_{\text{REP}}(\mathbf{H}) = (m_1, \dots, m_{q^r-1}) \in \mathbb{N}^{q^r-1}$, where m_j is the number of times that the j -th element of $(\mathbb{F}_q^r)^*$ (ordered w.r.t. any ordering) appears among the columns of $\mathbf{H}$. We define the integer $N_{\text{swaps}}(\mathbf{H})$ as

$$N_{\text{swaps}}(\mathbf{H}) = \prod_{i=1}^{q^r-1} m_i! \quad (16)$$

Proposition 4. If $\pi(\mathbf{V})\mathbf{H}^\top = \mathbf{E}$, then from $\pi \in \text{Sym}(n)$ (or equivalently from $\pi(\mathbf{V})$) we can obtain, performing only swaps, $N_{\text{swaps}}(\mathbf{H})$ solutions to the problem (including π).

Proof. Every swap of the permutation indices corresponding to two repeating columns gives a new distinct solution. Equation (16) counts the number of all possible swaps combined. $\square$

Proposition 5. Let $\mathbf{H} \in \mathbb{F}_q^{r \times n}$ be a matrix with no zero columns and with $\text{COL}_{\text{REP}}(\mathbf{H}) = (m_1, \dots, m_{q^r-1})$. Let $x \leq n$ be a positive integer and let

$$J_{n,r,q}^{(x)} = \left\{ (\mu_1, \dots, \mu_{q^r-1}) \in \mathbb{N}^{q^r-1} \mid \mu_1 + \dots + \mu_{q^r-1} = x, 0 \leq \mu_i \leq m_i \right\}.$$

Then

$$|\mathcal{S}_x(\mathbf{H})| = \sum_{(\mu_1, \dots, \mu_{q^r-1}) \in J_{n,r,q}^{(x)}} \frac{x!}{\mu_1! \cdots \mu_{q^r-1}!}. \quad (17)$$

Proof. $|\mathcal{S}_x(\mathbf{H})|$ is given by the possible ways of choosing ordered collections of x elements from n with no replacement, but some of them might be repeated. This follows the multinomial distribution, which gives us Equation (17). Notice that if $\mathbf{H}$ has no repeated columns, $\text{COL}_{\text{REP}}(\mathbf{H})$ has n entries equal to 1 and the rest of them equal to 0, so $|J_{n,r,q}^{(x)}| = \binom{n}{x}$, which implies

$$\sum_{(\mu_1, \dots, \mu_{q^r-1}) \in J_{n,r,q}^{(x)}} \frac{x!}{\mu_1! \cdots \mu_{q^r-1}!} = \binom{n}{x} x! = \frac{n!}{(n-x)!},$$

which is what we expected. $\square$

Remark 13. Since $\bar{\mathbf{H}}$ is modeled as random, we assume that its columns are uniformly distributed over $(\mathbb{F}_q^d)^*$, that is $\text{COL}_{\text{REP}}(\bar{\mathbf{H}}) = (u_1, \dots, u_{q^d-1})$, where

$(u_1, \dots, u_{q^d-1}) \in \mathbb{N}^{q^d-1}$ is an element that minimizes the Euclidean distance from $(\frac{w}{q^d-1}, \dots, \frac{w}{q^d-1})$ in the set $\{(v_1, \dots, v_{q^d-1}) \in \mathbb{N}^{q^d-1} \mid \sum_{i=1}^{q^d-1} v_i = w\}$. Notice that this element is not unique: if $(u_1, \dots, u_{q^d-1})$ satisfies these requirements, then any permutation of it does as well. To justify this assumption, we model the distribution of each entry of $\text{COL}_{\text{REP}}(\bar{\mathbf{H}})$ as a binomial random variable with parameters (w, p) , where $p = \frac{1}{q^d-1}$ is the probability of selecting a specific column (the probability of “success”). Hence, each entry has expected value $w \cdot p = \frac{w}{q^d-1}$, and by requiring every entry to be an integer, we obtain the above estimate.

Let $\text{COL}_{\text{REP}}^{(u)}(n, r, q)$ denote the vector $(u_1, \dots, u_{q^r-1}) \in \mathbb{N}^{q^r-1}$ as in Remark 13 and let $N_{\text{swaps}}^{(u)}(n, r, q) := \prod_{i=1}^{q^r-1} u_i!$. We also define $M_{\text{REP}}(n, r, q, x)$ as $|\mathcal{S}_x(\mathbf{H})|$, where $\mathbf{H} \in \mathbb{F}_q^{r \times n}$ is random with no zero columns, and therefore $M_{\text{REP}}(n, r, q, x)$ is computed using Proposition 5 with $\text{COL}_{\text{REP}}(\mathbf{H}) = \text{COL}_{\text{REP}}^{(u)}(n, r, q)$.

The next proposition presents the average running time of SBC^* (Algorithm 5) and $\text{SBC}_{\text{dual}}^*$ (Algorithm 6) in the case where $\mathbf{V}$ may contain repeated columns. Notice that, in our setting, the matrix that may have repeated columns is $\bar{\mathbf{H}}$; however, since we are solving the dual instance, it plays the role of $\mathbf{V}$ in Algorithm 5 and Algorithm 6.

Proposition 6. *Let $\mathbf{H} \in \text{GL}_{r,n}(\mathbb{F}_q)$ with no repeated columns and s.t. its last row is a vector of all ones, $\mathbf{V} \in \text{GL}_{\ell,n}(\mathbb{F}_q)$ be a matrix that might have repeated columns but no zero columns (and $\mathbf{H}, \mathbf{V}$ can be modeled as a random matrix with these properties) and $\mathbf{E} \in \mathbb{F}_q^{\ell \times r}$. Let now $w_1, w_2, d \in \mathbb{N}^*$ be valid parameters for SBC^* , that is $w = w_1 + w_2 \leq n$ and $N_{n,r}(w, d) > 1$. Then Algorithm 5 runs for $d \leq \tilde{r} \leq r$ in average time*

$$T_{\text{SBC}^*}(n, r, q, \ell, w_1, w_2, d, \tilde{r}) = T_{\text{ISD}}^{(d)}(n, r, w) + T_{\mathcal{K}}^* + T_{\mathcal{L}}^* + |\mathcal{L}^*|, \quad (18)$$

and Algorithm 6 runs for $0 \leq \tilde{r} \leq \ell + 1$ in average time

$$T_{\text{SBC}_{\text{dual}}^*}(n, r, q, \ell, w_1, w_2, d, \tilde{r}) = T_{\text{ISD}}^{(d)}(n, r, w) + T_{\mathcal{K}} + \bar{T}_{\mathcal{L}}, \quad (19)$$

with

$$\begin{aligned} T_{\mathcal{K}}^* &:= \sum_{\bar{w} \in \{w_1, w_2\}} M_{\text{REP}}(n, \ell, q, \bar{w}) + \frac{1}{q^{d\ell}} \cdot \prod_{\bar{w} \in \{w_1, w_2\}} M_{\text{REP}}(n, \ell, q, \bar{w}), \\ T_{\mathcal{L}}^* &:= M_{\text{REP}}(n, \ell, q, n - r - w + \tilde{r}) + |\mathcal{K}^*| + \frac{M_{\text{REP}}(n, \ell, q, n - r - w + \tilde{r}) \cdot |\mathcal{K}^*|}{q^{\ell(\tilde{r}-d)}}, \\ \bar{T}_{\mathcal{L}} &:= \bar{T}_{\mathcal{L}}(n, \ell, w, \tilde{r}, |\bar{\mathcal{K}}|) \text{ as in Equation (14)}, \\ |\mathcal{K}^*| &:= \frac{M_{\text{REP}}(n, \ell, q, w)}{q^{d\ell}}, \quad |\bar{\mathcal{K}}| := |\mathcal{K}^*| \cdot N_{\text{swaps}}^{(u)}(w, \ell, q), \\ |\mathcal{L}^*| &:= \frac{M_{\text{REP}}(n - w, \ell, q, n - r - w + \tilde{r}) \cdot |\mathcal{K}^*|}{q^{\ell(\tilde{r}-d)}}. \end{aligned}$$

Proof. The running time of Algorithm 5 is obtained in the same way as for Algorithm 3. Thus, we obtain Equation (18) by adapting Proposition 2 to the case in which $\mathbf{V}$ has repeated columns. Indeed, we obtain $T_{\mathcal{K}}^*$, $T_{\mathcal{L}}^*$, $|\mathcal{K}^*|$ and $|\mathcal{L}^*|$ as in Proposition 2 but we estimate $\mathcal{S}_x(\mathbf{V})$ with Equation (17) instead of $\frac{n!}{(n-x)!}$. The same reasoning also applies to Algorithm 6, but to prove Equation (19) we still need to clarify $|\bar{\mathcal{K}}|$. As already mentioned, if $\mathbf{V}$ has repeated columns, then InvCand returns more than one candidate. Since $\mathbf{V}$ is modeled as a random matrix, we have that every candidate $\mathbf{X} \in \mathcal{K}^*$ for $\sigma(\tilde{\mathbf{V}}_J)$ (with $|J| = w$), where $\mathcal{K}^*$ is the list obtained after the *filtering step*, generates $N_{\text{swaps}}^{(u)}(w, l, q)$ candidates for $\pi^{-1}(\mathbf{H})_I$ (the set I is possibly different for every $\mathbf{X}$). Therefore, Lines 7-14 in Algorithm 4 are executed $|\mathcal{K}^*| \cdot N_{\text{swaps}}^{(u)}(w, l, q)$ times, which gives the list cardinality. $\square$

Proposition 7. *Under the same hypothesis and notation as in Proposition 6, with $\mathbf{E} \in \mathbb{F}_q^{\ell \times r}$ being of the form $\mathbf{E} = [\mathbf{0}_{\ell \times (r-1)} \sum_{i=1}^n \mathbf{v}_i]$, we have that the average number of solutions returned respectively by Algorithm 5 and Algorithm 6 is given by*

$$\begin{aligned} N_{\text{SBC}^*}(n, r, q, \ell, w_1, w_2, d, \tilde{r}) &= |\mathcal{L}^*| \cdot \min \left\{ \frac{(r - \tilde{r})!}{q^{\ell(r - \tilde{r})} N_{\text{swaps}}^{(u)}(r - \tilde{r}, \ell, q)}, 1 \right\}, \\ N_{\text{SBC}_{\text{dual}}^*}(n, r, q, \ell, w_1, w_2, d, \tilde{r}) &= \frac{|\bar{\mathcal{K}}|(n - (w_1 + w_2))!}{(\ell + 1 - \tilde{r})! q^{\tilde{r}(r-1)}} \cdot \min \left\{ \frac{(\ell + 1 - \tilde{r})!}{q^{(r-1)(\ell+1-\tilde{r})}}, 1 \right\}. \end{aligned} \quad (20)$$

Proof. The number of solutions returned by Algorithm 5 is given by the number of elements in the list $\mathcal{L}$ that satisfy the condition at Line 5. Since, with the same notation as in Algorithm 5, $\mathbf{X}'$ can be modeled as a random $\ell \times (r - \tilde{r})$ matrix, then such condition is met with probability $\min \left\{ \frac{|\mathcal{S}_{r-\tilde{r}}(\mathbf{V}_U)|}{q^{\ell(r-\tilde{r})}}, 1 \right\}$ where U is the set of the indexes of the columns of $\mathbf{V}$ that are not columns of $\mathbf{X}$ (so $|U| = r - \tilde{r}$), therefore $|\mathcal{S}_{r-\tilde{r}}(\mathbf{V}_U)| = \frac{(r-\tilde{r})!}{N_{\text{swaps}}^{(u)}(r-\tilde{r}, \ell, q)}$.

On the other hand, the number of solutions returned by $\text{SBC}_{\text{dual}}^*$ is given by the number of candidates $\mathbf{W}$ of the form given at Line 9 of Algorithm 4 that satisfy both the condition at Line 10 and the condition at Line 12. The probability of the first condition to be met is the probability of a collision between two random $(r-1) \times \tilde{r}$ matrices. Considering the notation of Algorithm 4, the equality at Line 10 is checked for the matrices $\mathbf{W}\mathbf{V}'^\top$ and $\mathbf{E}'$, which are actually $r \times \tilde{r}$. However, the last row of such matrices is not random at all and, indeed, it always collides. To see this, let us recall that the last row of $\mathbf{H}$, and so the last one of $\mathbf{W}$, is a row of ones. On the other hand, the matrix $\mathbf{E}'$ is of the form

$$\mathbf{E}' = \mathbf{E}^{\text{ext}}(\mathbf{M}^\top)_{\{1, \dots, \tilde{r}\}} = \begin{bmatrix} \mathbf{0}_{(r-1) \times \ell} & \sum_{i=1}^n \mathbf{h}_i \\ (\sum_{i=1}^n \mathbf{v}_i)^\top & \end{bmatrix} (\mathbf{M}^\top)_{\{1, \dots, \tilde{r}\}},$$

since $\mathbf{E}^{\text{ext}}$ is as in Line 2 of Algorithm 6. Notice that the last entry of the last row of $\mathbf{E}^{\text{ext}}$ is given by $n \bmod q$ (which we denote with n_q). If we denote

$\mu(\mathbf{V}^{\text{ext}})$ with $\tilde{\mathbf{V}}_{\mu}^{\text{ext}}$, we have that $\bar{\mathbf{V}}^{\top} = (\tilde{\mathbf{V}}_{\mu}^{\text{ext}\top})_{\{1, \dots, n-(\ell+1)+\tilde{r}\}} (\mathbf{M}^{\top})_{\{1, \dots, \tilde{r}\}}$ and, since the last row of $\mathbf{W}$ is a row of ones, the last row of $\mathbf{W}\bar{\mathbf{V}}^{\top}$ is given by $\left(\sum_{i=1}^{n-(\ell+1)+\tilde{r}} (\tilde{\mathbf{v}}_{\mu}^{\text{ext}})_i\right)^{\top} (\mathbf{M}^{\top})_{\{1, \dots, \tilde{r}\}}$. On the other hand, the last row of $\mathbf{E}'$ is given by

$$\begin{aligned} [\sum_{i=1}^n \mathbf{v}_i \ n_q] (\mathbf{M}^{\top})_{\{1, \dots, \tilde{r}\}} &= \left(\sum_{i=1}^n (\tilde{\mathbf{v}}_{\mu}^{\text{ext}})_i\right)^{\top} (\mathbf{M}^{\top})_{\{1, \dots, \tilde{r}\}} \\ &= \left(\sum_{i=1}^{n-(\ell+1)+\tilde{r}} (\tilde{\mathbf{v}}_{\mu}^{\text{ext}})_i\right)^{\top} (\mathbf{M}^{\top})_{\{1, \dots, \tilde{r}\}}, \end{aligned}$$

where the last equality follows from the fact that, by construction, $(\tilde{\mathbf{v}}_{\mu}^{\text{ext}})_i^{\top} (\mathbf{M}^{\top})_{\{1, \dots, \tilde{r}\}} = \mathbf{0}$ for every i s.t. $n - (\ell + 1) + \tilde{r} + 1 \leq i \leq n$. To prove Equation (20), it remains to estimate the probability of the condition at Line 12 to be met. This is the probability of the matrix $\mathbf{W}'$ having as columns those of $\mathbf{H}$ that do not appear in $\mathbf{W}_{\{1, \dots, n-(\ell+1)\}}$. This event happens for sure for $\tilde{r}$ columns (because $\mathbf{W}$ satisfies the condition at Line 10), while the other $\ell + 1 - \tilde{r}$ columns can be considered random. Therefore, the probability is given by $\min \left\{ \frac{|\mathcal{S}_{\ell+1-\tilde{r}}(\mathbf{H}_U)|}{q^{(r-1)(\ell+1-\tilde{r})}}, 1 \right\}$, where U is the set of columns of $\mathbf{H}$ that don't appear in $\mathbf{W}$. Since $\mathbf{H}$ has no repeated columns, $|\mathcal{S}_{\ell+1-\tilde{r}}(\mathbf{H}_U)| = (\ell + 1 - \tilde{r})!$. Notice that again we consider collision between random matrices with $r - 1$ rows, since, analogously as above, one can show that the last row of $\mathbf{W}'$ is always a row of ones. $\square$

Remark 14. We showed in Proposition 7 how extending the matrix $\mathbf{H}$ in Algorithm 6 does not help to reduce the number of collisions. The same reasoning applies to the matrix $\mathbf{V}$ in Algorithm 5, hence it is not useful to extend it. Moreover, one could observe that in Algorithm 5 and Algorithm 6 the matrices that are being checked for collisions have a column deriving from the extension of $\mathbf{H}$ and $\mathbf{V}$, respectively, but they are still considered random. Indeed, we consider the matrices at Line 5 in Algorithm 5 to be random $\ell \times (r - \tilde{r})$ matrices instead of $\ell \times (r - 1 - \tilde{r})$ (the same applies to Algorithm 6 with $(r - 1) \times (\ell + 1 - \tilde{r})$ instead of $(r - 1) \times (\ell - \tilde{r})$). We can do this because the “non-random” column appears in a matrix that is being multiplied on the right by another matrix, so the resulting product cannot be split as it was done Proposition 7 for the last row: roughly speaking, the “row-times-columns” multiplication scrambles the column-structure of the matrix on the left. Let us consider the matrix $\mathbf{X}'$ in Algorithm 5 (for Algorithm 6 the reasoning is analogous), which is obtained as $\bar{\mathbf{E}} - \mathbf{X}_{\{1, \dots, n-r\}} \bar{\mathbf{U}}^{\top}$. We have that

$$\begin{aligned} \mathbf{X}_{\{1, \dots, n-r\}} \bar{\mathbf{U}}^{\top} &= \mathbf{X}_{\{1, \dots, n-r\}} \left((\mathbf{M}\sigma(\mathbf{H}))_{\{1, \dots, n-r\}}^{\{\tilde{r}+1, \dots, r\}} \right)^{\top} \\ &= \mathbf{X}_{\{1, \dots, n-r\}} (\sigma(\mathbf{H})^{\top})_{\{1, \dots, n-r\}}^{\{\tilde{r}+1, \dots, r\}} \mathbf{M}_{\{\tilde{r}+1, \dots, r\}}^{\top} \\ &= [\mathbf{A} \quad \sum_{i=1}^{n-r} \mathbf{x}_i] \mathbf{M}_{\{\tilde{r}+1, \dots, r\}}^{\top}, \end{aligned}$$

for some $\mathbf{A} \in \mathbb{F}_q^{\ell \times (r-1)}$. Since $\bar{\mathbf{E}} = \mathbf{E} \mathbf{M}_{\{\tilde{r}+1, \dots, r\}}^\top$, we have that

$$\begin{aligned} \mathbf{X}' &= \bar{\mathbf{E}} - \mathbf{X}_{\{1, \dots, n-r\}} \bar{\mathbf{U}}^\top = ([\mathbf{0} \quad \sum_{i=1}^n \mathbf{v}_i] - [\mathbf{A} \quad \sum_{i=1}^{n-r} \mathbf{x}_i]) \mathbf{M}_{\{\tilde{r}+1, \dots, r\}}^\top \\ &= [-\mathbf{A} \quad \sum_{i=1}^r \bar{\mathbf{x}}_i] \mathbf{M}_{\{\tilde{r}+1, \dots, r\}}^\top, \end{aligned} \quad (21)$$

where $\{\bar{\mathbf{x}}_i\}_{i=1, \dots, r}$ is the set of the columns of $\mathbf{V}$ that do not appear in $\mathbf{X}$. Note that these are the columns of $\mathbf{X}'$ if $\mathbf{X}$ is the candidate that leads to the solution. However, we cannot express the matrix resulting from the product in Equation (21) in term of the last column of the matrix on the left, which is the one that we just showed to be not random. Therefore, for the sake of our analysis, we consider this matrix to look random.

Finally, notice that a similar reasoning can be applied when checking for collisions at Line 13 of Algorithm 3 (when using Algorithm 5) and at Line 10 of Algorithm 4 (when using Algorithm 6), for the case in which, respectively, $\tilde{r} = r$ and $\tilde{r} = \ell + 1$. Indeed, in these cases we consider collisions between random $\ell \times (r - d)$ and $(r - 1) \times (\ell + 1)$ matrices (instead of between $\ell \times (r - 1 - d)$ and $(r - 1) \times \ell$ matrices), respectively. Notice that the above considerations apply only when the IPKP instance arises from the dehomogenization of a PKP instance, which is why this assumption is required in Proposition 7. For generic IPKP instances, the algorithm behaves identically, but the number of solutions can be estimated without these considerations, resulting in slightly smaller complexity estimates.

Remark 15. In Proposition 6 we do not require the existence of a solution. We do this since we use this proposition to estimate the complexity of solving the subinstances, which are not required to have a solution, except for one of them. Therefore to estimate the running time of SBC^* and $\text{SBC}_{\text{dual}}^*$ applied on $\binom{n}{w} - 1$ subinstances we use respectively Equation (18) and Equation (19), while in one subinstance we use $T_{\text{SBC}^*}^{(1)}$ and $T_{\text{SBC}_{\text{dual}}^*}^{(1)}$. These are obtained in the same way as in Proposition 7, but taking $|\mathcal{K}^*| = \max \left\{ \frac{\text{MREP}(n, \ell, q, w)}{q^{d\ell}}, 1 \right\}$ instead of taking it as in Proposition 6. Analogously, we define $N_{\text{SBC}^*}^{(1)}$ and $N_{\text{SBC}_{\text{dual}}^*}^{(1)}$.

4.3 New algorithm

We can now define our complete algorithm for solving IPKP, which incorporates all the modifications described above. The algorithm takes two additional Boolean parameters, $\mathbf{v}_{\text{inn}}$ and $\mathbf{v}_{\text{out}}$, which specify whether Algorithm 4 is used respectively in the subinstances and in the outer instance. Here, the algorithm $\text{InvCand}^*(\mathbf{X})$ takes as input $\mathbf{X} \in \mathcal{S}_w(\bar{\mathbf{H}})$, a solution to the subinstance (that is, using the notation of the algorithm, $\mathbf{X}(\mathbf{W}^{\text{ext}})^\top = \mathbf{E}^{*\text{ext}}$), and returns all corresponding candidates for $\bar{\mathbf{W}}^{\text{ext}}$. In general, there is more than one such candidate, since $\bar{\mathbf{H}}$ typically contains repeated columns.

Lemma 1. *Let $\mathbf{H} \in \text{GL}_{r,n}$, $\mathbf{V} \in \text{GL}_{\ell,n}$ and $\mathbf{E} \in \mathbb{F}_q^{\ell \times r}$ be matrices that define an IPKP-instance obtained by de-homogenizing an $(n, r - 1, q, \ell)$ - PKP instance*

Algorithm 7 NewAlgorithm($\mathbf{H}, \mathbf{V}, \mathbf{E}$)

Parameters: $v_{\text{inn}}, v_{\text{out}} \in \{0, 1\}$, $w, \tilde{r}, d \in \mathbb{N}$ with $w \leq n$, $N_{n,r}(w, d) > 1$ and $0 \leq \tilde{r} \leq \ell + 1$ **if** v_{out} **else** $d \leq \tilde{r} \leq r$.

Input: $\mathbf{H} \in \text{GL}_{r,n}(\mathbb{F}_q)$, $\mathbf{V} \in \text{GL}_{\ell,n}(\mathbb{F}_q)$, $\mathbf{E} \in \mathbb{F}_q^{\ell \times r}$,

Output: $\pi \in \text{Sym}(n)$ such that $\pi(\mathbf{V})\mathbf{H}^\top = \mathbf{E}$

- 1: Use ISD to find a subcode of the code generated by $\mathbf{H}$ with dimension d and support size $w = w_1 + w_2$. Call $\mathcal{C}$ this subcode, $\mathbf{H}^*$ its generator matrix and $\mathbf{S} \in \text{GL}_{d,r}(\mathbb{F}_q)$ the matrix s.t. $\mathbf{H}^* = \mathbf{S}\mathbf{H}$.
- 2: Compute $\sigma \in \text{Sym}(n)$ such that $\text{Supp}(\sigma(\mathcal{C})) = \{n-r+\tilde{r}-w+1, \dots, n-r+\tilde{r}\}$
- 3: $J \leftarrow \{n-r+\tilde{r}-w+1, \dots, n-r+\tilde{r}\}$
- 4: $\mathbf{E}^* \leftarrow \mathbf{E}\mathbf{S}^\top$, $\tilde{\mathbf{H}} \leftarrow \sigma(\mathbf{H}^*)_J$
- 5: Initialize an empty list $\mathcal{K}$
- 6: **for** $\mathbf{W} \in \tilde{\mathcal{S}}_w(\mathbf{V})$ **do**
- 7: $\mathbf{W}^{\text{ext}} \leftarrow \begin{bmatrix} \mathbf{W} \\ (1, \dots, 1) \end{bmatrix}$, $\mathbf{E}^{*\text{ext}} \leftarrow [\mathbf{E}^{*\top} \sum_{i=1}^w \tilde{\mathbf{h}}_i]$
- 8: **if** v_{inn} **then**
- 9: $\mathcal{K} \leftarrow \mathcal{K} \cup \text{SBC}_{\text{dual}}^*(\mathbf{W}^{\text{ext}}, \tilde{\mathbf{H}}, \mathbf{E}^{*\text{ext}})$
- 10: **else**
- 11: $\mathcal{K}^* \leftarrow \text{SBC}^*(\mathbf{W}^{\text{ext}}, \tilde{\mathbf{H}}, \mathbf{E}^{*\text{ext}})$
- 12: **for** $\mathbf{X} \in \mathcal{K}^*$ **do**
- 13: $\mathcal{K} \leftarrow \mathcal{K} \cup \text{InvCand}^*(\mathbf{X})$
- 14: **if** v_{out} **then**
- 15: $\mathbf{V}^{\text{ext}} \leftarrow \begin{bmatrix} \mathbf{V} \\ (1, \dots, 1) \end{bmatrix}$, $\mathbf{E}^{\text{ext}} \leftarrow [\mathbf{E}^\top \sum_{i=1}^n \mathbf{h}_i]$
- 16: $\tilde{\mathbf{H}} \leftarrow \text{DualGuessSolve}(\mathbf{H}, \mathbf{V}^{\text{ext}}, \mathbf{E}^{\text{ext}}, \mathcal{K})$
- 17: Compute $\pi \in \text{Sym}(n)$ s.t. $\pi(\mathbf{H}) = \tilde{\mathbf{H}}$
- 18: **Return:** π^{-1}
- 19: **else**
- 20: Proceed as in Lines 8-19 of Algorithm 3

that has a solution. Let $v_{\text{inn}} \in \{0, 1\}$ be a boolean parameter that specifies whether to use Algorithm 6 or Algorithm 5 in the (inner) subinstances. Let $w, d \in \mathbb{N}^*$ be valid parameters for the outer instance, that is $w \leq n$ and $N_{n,r}(w, d) > 1$. Let $\ell^* = \min\{\ell, w-d+1\}$ and let $w_1^*, w_2^*, d^*, \tilde{r}^*$ be valid parameters for the subinstances, that is $w^* = w_1^* + w_2^* \leq w$, $N_{w, \ell^*+1}(w^*, d^*) > 1$ and

$$\begin{cases} 0 \leq \tilde{r}^* \leq d+1 & \text{if } v_{\text{inn}} \\ d^* \leq \tilde{r}^* \leq \ell^*+1 & \text{otherwise} \end{cases}.$$

We denote the vector of the parameters used in the subinstances as $\text{par}_{\text{inn}} = (w, \ell^*+1, q, d, w_1^*, w_2^*, d^*, \tilde{r}^*)$. Then the average running time of Lines 6-13 of Algorithm 7, that is the average running time of the modified filtering step, is

given by

$$T_{\mathcal{K}}^{(\mathbf{v}_{\text{inn}})} = \begin{cases} ((\binom{n}{w} - 1) T_{\text{SBC}^*_{\text{dual}}}(\text{par}_{\text{inn}}) + T_{\text{SBC}^*_{\text{dual}}}^{(1)}(\text{par}_{\text{inn}})), & \text{if } \mathbf{v}_{\text{inn}}, \\ ((\binom{n}{w} - 1) T_{\text{SBC}^*}(\text{par}_{\text{inn}}) + T_{\text{SBC}^*}^{(1)}(\text{par}_{\text{inn}})), & \text{otherwise.} \end{cases}$$

Proof. $\mathbf{v}_{\text{inn}} = 1$: Since $\mathbf{v}_{\text{inn}} = 1$, the cost of executing Lines 6-13, which is $T_{\mathcal{K}}^{(\mathbf{v}_{\text{inn}})}$, is given by $|\mathcal{S}_w(\mathbf{V})| = \binom{n}{w}$ multiplied by the running time of Algorithm 6 applied to the subinstances. However, as anticipated before, one of the subinstance is required to have one solution, therefore we estimate the running time of Algorithm 6 applied to this subinstance with $T_{\text{SBC}^*_{\text{dual}}}^{(1)}(\text{par}_{\text{inn}})$, while for all the other subinstances we use $T_{\text{SBC}^*_{\text{dual}}}(\text{par}_{\text{inn}})$.

$\mathbf{v}_{\text{inn}} = 0$: The proof is exactly the same as in the previous case with Algorithm 6 replaced by Algorithm 5. $\square$

Lemma 2. *Under the same hypothesis and notation as in Lemma 1, then the cardinality of the list $\mathcal{K}$ obtained after Line 13 of Algorithm 7 is given on average by*

$$|\mathcal{K}^{(\mathbf{v}_{\text{inn}})}| = \begin{cases} ((\binom{n}{w} - 1) N_{\text{SBC}^*_{\text{dual}}}(\text{par}_{\text{inn}}) + \max\{N_{\text{SBC}^*_{\text{dual}}}^{(1)}(\text{par}_{\text{inn}}), 1\}), & \text{if } \mathbf{v}_{\text{inn}}, \\ ((\binom{n}{w} - 1) N_{\text{SBC}^*}(\text{par}_{\text{inn}}) + \max\{N_{\text{SBC}^*}^{(1)}(\text{par}_{\text{inn}}), 1\}) \cdot N_{\text{swaps}}^{(u)}(w, d, q), & \text{otherwise.} \end{cases}$$

Proof. $\mathbf{v}_{\text{inn}} = 0$: $|\mathcal{K}^{(\mathbf{v}_{\text{inn}})}|$ is the cardinality of the list $\mathcal{K}$ containing all the candidates for $\sigma(\tilde{\mathbf{V}})_J$. This is given by the cardinality of the list $\mathcal{K}^*$ defined at Line 11 containing all the permuted $\tilde{\mathbf{H}}$ that solve the subinstance multiplied by $N_{\text{swaps}}^{(u)}(w, d, q)$, since, for every element $\mathbf{X} \in \mathcal{K}^*$, $\text{InvCand}^*(\mathbf{X})$ returns $N_{\text{swaps}}^{(u)}(w, d, q)$ candidates for $\sigma(\tilde{\mathbf{V}})_J$. Because of Proposition 7, we estimate $|\mathcal{K}^*|$ with $N_{\text{SBC}^*}(\text{par}_{\text{inn}})$ in $\binom{n}{w} - 1$ subinstances, while we use $\max\{N_{\text{SBC}^*}^{(1)}(\text{par}_{\text{inn}}), 1\}$ for one subinstance.

$\mathbf{v}_{\text{inn}} = 1$: The proof is obtained in the same way as in the previous case, but we don't have the factor $N_{\text{swaps}}^{(u)}(w, d, q)$ because $\text{SBC}^*_{\text{dual}}$ already returns the candidates for $\sigma(\tilde{\mathbf{V}})_J$ (the swaps are handled internally). $\square$

Lemma 3. *Under the same hypothesis and notation as in Lemma 1 and Lemma 2. Moreover, let $\mathbf{v}_{\text{out}} \in \{0, 1\}$ be a boolean parameter that specifies whether to use Algorithm 4 or the original **STEP 2-STEP 3** in the outer instance and let $\tilde{r} \in \mathbb{N}$ be a valid parameter for the outer instance, that is*

$$\begin{cases} 0 \leq \tilde{r} \leq \ell + 1 & \text{if } \mathbf{v}_{\text{out}} \\ d \leq \tilde{r} \leq r & \text{otherwise} \end{cases}.$$

Then the average running time of Lines 14-20 is given by

$$T_{\mathcal{L}}^{(\mathbf{v}_{\text{inn}}, \mathbf{v}_{\text{out}})} = \begin{cases} \bar{T}_{\mathcal{L}}(n, \ell, w, \tilde{r}, |\mathcal{K}^{(\mathbf{v}_{\text{inn}})}|), & \text{if } \mathbf{v}_{\text{out}} \\ \frac{n!}{(r+w-\tilde{r})!} + |\mathcal{K}^{(\mathbf{v}_{\text{inn}})}| + \frac{n!|\mathcal{K}^{(\mathbf{v}_{\text{inn}})}|}{(r+w-\tilde{r})!q^{\ell(\tilde{r}-d)}} + \frac{(n-w)!|\mathcal{K}^{(\mathbf{v}_{\text{inn}})}|}{(r-\tilde{r})!q^{\ell(\tilde{r}-d)}}, & \text{otherwise.} \end{cases}$$

Proof. $v_{\text{out}} = 0$: In these cases the proof follows as in Proposition 2 by using the estimates for $|\mathcal{K}|$ given by Lemma 2: in both cases, the first three terms of the sum are obtained as in Equation (12) by replacing $|\mathcal{K}|$ with $|\mathcal{K}^{(v_{\text{inn}})}|$. The last term of both sums is obtained in the same way from Equation (13).

$v_{\text{out}} = 1$: The proof follows from Proposition 3 and Lemma 2. $\square$

Theorem 1. *Under the same hypothesis and notation as in Lemma 1, Lemma 2 and Lemma 3, then the average running time of Algorithm 7 is given by*

$$T^{(4)}(n, r, q, \ell, w, d, \tilde{r}, w_1^*, w_2^*, d^*, \tilde{r}^*, v_{\text{inn}}, v_{\text{out}}) = T_{\text{ISD}}^{(d)}(n, r, w) + T_{\mathcal{K}}^{(v_{\text{inn}})} + T_{\mathcal{L}}^{(v_{\text{inn}}, v_{\text{out}})}.$$

Proof. The proof follows by combining Equation (3), Lemma 1 and Lemma 3, since the cost of Lines 2-5 is negligible. $\square$

Remark 16. Actually, in Theorem 1 we consider n and r to be valid values for respectively w and d , since the code generated by $\mathbf{H}$ is a subcode of itself. We make this explicit since n and r might not satisfy $N_{n,r}(n, r) > 1$. In this case Algorithm 7 becomes, if $v_{\text{out}} = 0$, the same as Algorithm 3, as we are applying SBC^* to a single instance (since $\binom{n}{n} = 1$) with parameters (n, r, q, ℓ) , where $\mathbf{V}$ has no repeated columns. Consequently, in this setting, the running time of SBC^* coincides with that of SBC.

5 Discussion and Comparisons

In this section, we assess the impact of the algorithmic improvements introduced in Section 4 by comparing the complexity of Algorithm 7 with that of KMP and SBC.

Complexity for adaptive field size. Figure 3 reports the complexities of the three algorithms for $n \in \{64, 92, 118\}$, rates $R = \frac{r}{n} \in \{0.3, 0.5, 0.7\}$, and varying values of ℓ . For each parameter set (n, R, ℓ) , the field size q is chosen as the smallest power of a prime satisfying conditions i)–iv) in Section 2.1, following the same approach adopted by Santini *et al.* [24]. Notice that, for given n, R and ℓ , a smaller field size q generally leads to a higher estimated time complexity of the problem instance when solving it with KMP or SBC. Hence, this choice of considering the smallest q admissible is motivated by the fact that it allows the construction of instances with smaller public key sizes. Moreover, notice that these conditions i)–iv) in Section 2.1 have to be met by the parameters of the homogeneous instance, since by adding a row of ones to the parity check matrix and extending the zero matrix accordingly, we introduce equations that are verified by every permutation. For every n and R , we consider all values of ℓ such that the corresponding q is at least 3. In addition, for each parameter set analyzed, we consider the minimum complexity between the problem and its dual version, as defined in Equation (6). As explained in Remark 16, Algorithm 7 automatically falls back to the SBC setting whenever this is the most convenient

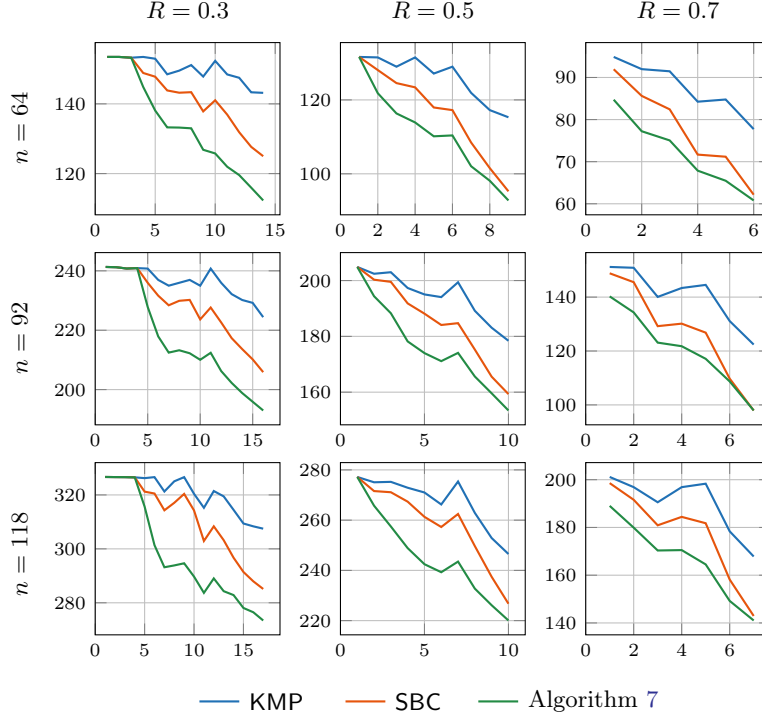


Fig. 3: Time complexity, expressed in $\log_2$ of the number of matrix and list operations, of KMP, SBC and Algorithm 7 for $n \in \{64, 92, 118\}$, $R \in \{0.3, 0.5, 0.7\}$ and different values of ℓ along the x -axis.

choice. Hence, as one can observe in the plots, its complexity is always smaller than or equal to that of SBC. In particular, for $R = 0.7$, we observe gains already from $\ell = 1$, whereas for $R = 0.5$ and $R = 0.3$, the improvement becomes evident only for $\ell = 2$ and $\ell = 4$, respectively. On the other hand, KMP is generally worse or equal than SBC and Algorithm 7.

Complexity for large fields and implications for PERK 1.0. When q is larger than the smallest suitable power of a prime, we observe that Algorithm 7 quickly converges to SBC. This behavior is expected, since larger field sizes q prevent the existence of subcodes with small support size that are useful when solving the IPKP subinstances, as explained in Section 4.2. Consequently, our variant of **STEP 1** in SBC becomes ineffective in this regime. This trend is clearly visible when comparing the complexities of Algorithm 7 and SBC for the PERK 1.0 [1] parameter sets, as reported in Table 2. We observe that Algorithm 7 achieves improvements for minimal field sizes q , while it coincides with SBC for the PERK field size $q = 1021$. This choice of q , despite leading to larger

public keys, was motivated by other constraints imposed by the MPC-in-the-Head framework used in PERK.

Discussion on PERK 2.1 parameter sets. We found that SBC remains optimal also for the parameter sets of the more recent PERK 2.1 [2]. Unlike in the previous version of PERK, the field size $q = 2048$ is close to the minimum admissible value according to conditions i)–iv) in Section 2.1. Specifically, the minimal field sizes satisfying these conditions are 1997, 2017, and 1993 for security levels I, III, and V, respectively. However, in this case, there is another reason supporting the optimality of SBC: across all parameter sets, we have $\ell = 1$ and $0.42 \leq R \leq 0.5$. From Figure 3, which reports the same values of n as in the PERK 2.1 parameter sets, we indeed observe that for this range of code rates and $\ell = 1$, Algorithm 7 matches SBC in complexity.

	(n, r, ℓ)	q	KMP	SBC	Algorithm 7
perk-i-3	(79, 35, 3)	1021	146.7	145.6	145.6
minimum q		13	180.1	175.4	166.0
perk-i-5	(83, 36, 5)	1021	153.8	147.7	147.7
minimum q		5	191.3	185.2	171.2
perk-iii-3	(112, 54, 3)	1021	212.7	210.8	210.8
minimum q		16	257.6	254.7	242.6
perk-iii-5	(116, 55, 5)	1021	219.0	212.6	212.6
minimum q		5	276.8	270.1	248.6
perk-v-3	(146, 75, 3)	1021	280.0	275.2	275.2
minimum q		16	342.5	339.2	325.5
perk-v-5	(150, 76, 5)	1021	280.9	274.1	274.1
minimum q		5	363.6	358.0	329.0

Table 2: Comparison of complexities for different PERK 1.0 parameter set and equivalent parameters but with minimal field size q .

Acknowledgments

We thank the anonymous reviewers for their insightful comments and suggestions that helped improving the manuscript. The third author acknowledges support from Ripple’s University Blockchain Research Initiative and from the INDAM group GNSAGA.

References

1. Aaraj, N., Bettaieb, S., Bidoux, L., Budroni, A., Dyseryn, V., Esser, A., Gaborit, P., Kulkarni, M., Mateu, V., Palumbi, M., Perin, L., Tillich, J.: PERK. Tech. rep., National Institute of Standards and Technology (2023), available at <https://csrc.nist.gov/Projects/pqc-dig-sig/round-1-additional-signatures>
2. Aaraj, N., Bettaieb, S., Bidoux, L., Budroni, A., Dyseryn, V., Esser, A., Feneuil, T., Gaborit, P., Kulkarni, M., Mateu, V., Palumbi, M., Perin, L., Rivain, M., Tillich, J.P., Xagawa, K.: PERK: A post-quantum signature scheme (version 2.1.0) (Sep 2025), specification document; available at <https://pqc-perk.org/assets/downloads/perk-v2.1.0.pdf>
3. Alagic, G., Bros, M., Ciadoux, P., Cooper, D., Dang, Q., Dang, T., Kelsey, J.M., Lichtinger, J., Miller, C.A., Moody, D., Peralta, R., Perlner, R., Robinson, A., Silberg, H., Smith-Tone, D., Waller, N., Liu, Y.K.: Status report on the first round of the additional digital signature schemes for the nist post-quantum cryptography standardization process. NIST Interagency/Internal Report (NISTIR) 8528, National Institute of Standards and Technology (NIST), Gaithersburg, MD (Oct 2024). <https://doi.org/10.6028/NIST.IR.8528>, <https://doi.org/10.6028/NIST.IR.8528>
4. Baldi, M., Battagliola, M., Mechri, R.E., Santini, P., Schiavoni, R., Zuane, D.D.: SPECK: Signatures from permutation equivalence of codes and kernels. Cryptology ePrint Archive, Paper 2025/923 (2025), <https://eprint.iacr.org/2025/923>
5. Baritaud, T., Campana, M., Chauvaud, P., Gilbert, H.: On the security of the permuted kernel identification scheme. In: Brickell, E.F. (ed.) CRYPTO'92. LNCS, vol. 740, pp. 305–311. Springer, Berlin, Heidelberg (Aug 1993). https://doi.org/10.1007/3-540-48071-4_21
6. Berger, T.P., Gueye, C.T., Klamti, J.B.: A np-complete problem in coding theory with application to code based cryptography. In: El Hajji, S., Nitaj, A., Souidi, E.M. (eds.) Codes, Cryptology and Information Security. pp. 230–237. Springer International Publishing, Cham (2017)
7. Beullens, W.: Sigma protocols for MQ, PKP and SIS, and Fishy signature schemes. In: Canteaut, A., Ishai, Y. (eds.) EUROCRYPT 2020, Part III. LNCS, vol. 12107, pp. 183–211. Springer, Cham (May 2020). https://doi.org/10.1007/978-3-030-45727-3_7
8. Bidoux, L., Gaborit, P.: Compact post-quantum signatures from proofs of knowledge leveraging structure for the pkp, sd and rsd problems. In: El Hajji, S., Mesnager, S., Souidi, E.M. (eds.) Codes, Cryptology and Information Security. pp. 10–42. Springer Nature Switzerland, Cham (2023)
9. Budroni, A., Defranceschi, M., Pintore, F.: Accompanying repository. <https://github.com/Crypto-TII/pkp-complexity> (2025), accessed: 2025-10-30
10. Faugère, J.C., Koussa, E., Macario-Rat, G., Patarin, J., Perret, L.: PKP-based signature scheme. Cryptology ePrint Archive, Report 2018/714 (2018), <https://eprint.iacr.org/2018/714>
11. Feneuil, T.: Building MPCitH-based signatures from MQ, MinRank, rank SD and PKP. Cryptology ePrint Archive, Report 2022/1512 (2022), <https://eprint.iacr.org/2022/1512>
12. Garey, M.R., Johnson, D.S.: Computers and Intractability; A Guide to the Theory of NP-Completeness. W. H. Freeman & Co., USA (1990)
13. Georgiades, J.: Some remarks on the security of the identification scheme based on permuted kernels. Journal of Cryptology **5**(2), 133–137 (Jan 1992). <https://doi.org/10.1007/BF00193565>

14. Jaulmes, É., Joux, A.: Cryptanalysis of PKP: A new approach. In: Kim, K. (ed.) PKC 2001. LNCS, vol. 1992, pp. 165–172. Springer, Berlin, Heidelberg (Feb 2001). https://doi.org/10.1007/3-540-44586-2_12
15. Koussa, E., Macario-Rat, G., Patarin, J.: On the complexity of the permuted kernel problem. Cryptology ePrint Archive, Report 2019/412 (2019), <https://eprint.iacr.org/2019/412>
16. LAMPE, R., PATARIN, J.: Analysis of some natural variants of the PKP algorithm. Cryptology ePrint Archive, Report 2011/686 (2011), <https://eprint.iacr.org/2011/686>
17. Lyubashevsky, V., Ducas, L., Kiltz, E., Lepoint, T., Schwabe, P., Seiler, G., Stehlé, D., Bai, S.: CRYSTALS-DILITHIUM. Tech. rep., National Institute of Standards and Technology (2022), available at <https://csrc.nist.gov/Projects/post-quantum-cryptography/selected-algorithms-2022>
18. National Institute of Standards and Technology: Post-Quantum Cryptography Standardization. <https://csrc.nist.gov/projects/post-quantum-cryptography> (2017)
19. National Institute of Standards and Technology: Post-quantum cryptography: Digital signature schemes. Round 2 Additional Signatures (2023), <https://csrc.nist.gov/projects/pqc-dig-sig/round-2-additional-signatures>
20. Patarin, J., Chauvaud, P.: Improved algorithms for the permuted kernel problem. In: Stinson, D.R. (ed.) CRYPTO’93. LNCS, vol. 773, pp. 391–402. Springer, Berlin, Heidelberg (Aug 1994). https://doi.org/10.1007/3-540-48329-2_33
21. Prest, T., Fouque, P.A., Hoffstein, J., Kirchner, P., Lyubashevsky, V., Pornin, T., Ricosset, T., Seiler, G., Whyte, W., Zhang, Z.: FALCON. Tech. rep., National Institute of Standards and Technology (2022), available at <https://csrc.nist.gov/Projects/post-quantum-cryptography/selected-algorithms-2022>
22. Sanna, C.: On the number of solutions to a random instance of the permuted kernel problem. Journal of Complexity **86**, 101898 (2025). <https://doi.org/https://doi.org/10.1016/j.jco.2024.101898>, <https://www.sciencedirect.com/science/article/pii/S0885064X2400075X>
23. Santini, P., Baldi, M., Chiaraluce, F.: A novel attack to the permuted kernel problem. In: 2022 IEEE International Symposium on Information Theory (ISIT). p. 1441–1446. IEEE Press (2022). <https://doi.org/10.1109/ISIT50566.2022.9834867>, <https://doi.org/10.1109/ISIT50566.2022.9834867>
24. Santini, P., Baldi, M., Chiaraluce, F.: Computational hardness of the permuted kernel and subcode equivalence problems. IEEE Trans. Inf. Theor. **70**(3), 2254–2270 (Mar 2024). <https://doi.org/10.1109/TIT.2023.3323068>, <https://doi.org/10.1109/TIT.2023.3323068>
25. Schwabe, P., Avanzi, R., Bos, J., Ducas, L., Kiltz, E., Lepoint, T., Lyubashevsky, V., Schanck, J.M., Seiler, G., Stehlé, D., Ding, J.: CRYSTALS-KYBER. Tech. rep., National Institute of Standards and Technology (2022), available at <https://csrc.nist.gov/Projects/post-quantum-cryptography/selected-algorithms-2022>
26. Shamir, A.: An efficient identification scheme based on permuted kernels (extended abstract) (rump session). In: Brassard, G. (ed.) CRYPTO’89. LNCS, vol. 435, pp. 606–609. Springer, New York (Aug 1990). https://doi.org/10.1007/0-387-34805-0_54
27. Shor, P.: Algorithms for quantum computation: discrete logarithms and factoring. In: Proceedings 35th Annual Symposium on Foundations of Computer Science. pp. 124–134 (1994). <https://doi.org/10.1109/SFCS.1994.365700>

A On the rank of the puncturings of $\mathbf{V}$

We explain here why we can assume that $\ell^* = \text{rank}(\mathbf{V}_W) = \min\{\ell, w - d + 1\}$. With the same notation as Section 4.2, we have that for W corresponding to the solution, $\text{rank}(\mathbf{V}_W) \leq \min\{\ell, w - d + 1\}$. Indeed, $\mathbf{V}_W$ is $\ell \times w$ and there exists $\tilde{\pi} \in \text{Sym}(w)$ such that

$$\tilde{\pi}(\mathbf{V}_W)\tilde{\mathbf{H}}^\top = \mathbf{E}^*.$$

Then, by rank-nullity theorem and by noting that $\text{rank}(\tilde{\pi}(\mathbf{V}_W)) = \text{rank}(\mathbf{V}_W)$ for any $\tilde{\pi} \in \text{Sym}(w)$, we have that

$$\text{rank}(\mathbf{V}_W) = \text{rank}(\tilde{\pi}(\mathbf{V}_W)) \leq w - \text{rank}(\tilde{\mathbf{H}}) + \text{rank}(\mathbf{E}^*) = w - d + 1. \quad (22)$$

We can safely assume the equality in Equation (22), that is $\text{rank}(\mathbf{V}_W) = \min\{\ell, w - d + 1\}$. Indeed, for ℓ small enough (which corresponds to the cases of cryptographic interest), we have $\ell < w - d + 1 \leq w$. Then, Equation (22) is satisfied, and $\mathbf{V}_W$ is a non-square submatrix of the full-rank matrix $\mathbf{V}$, and it is full rank with high probability. If, on the other hand, $\ell > w - d + 1$, we can obtain an equivalent instance after removing the redundant rows of $\mathbf{V}_W$, which are $\ell - (w - d + 1)$, and the resulting matrix is full-rank with high probability. It remains to consider the case $\ell = w - d + 1$. If $d > 1$, $\mathbf{V}_W$ is again non-square, so the previous considerations apply. Otherwise, if $d = 1$ and $\ell = w$, $\mathbf{V}_W$ is square, and the probability that it is not full rank is approximately $1/q$. In our estimates, however, we can safely assume that this never occurs. Indeed, when a square $\mathbf{V}_W$ is not full rank, solving such a subinstance requires an overhead of guessing $\tilde{\pi}$ on $w - \text{rank}(\mathbf{V}_W)$ columns. With high probability, this overhead appears in few cases and, when it appears, it is absorbed by the overall complexity, so we assume it to be negligible.